

Agentic AI Mastery Handbook

Chapter 2 – Tool Calling

Part 1 – Function Calling, Tool Schemas & Complete Tool-Calling Flow

Interview Importance: ★ ★ ★ ★ ★

Difficulty: Beginner → Intermediate

Prerequisites: Chapter 1 – Agentic AI Fundamentals

1. What is Tool Calling?



Tool calling is the mechanism through which an LLM can request that an external function or service be executed with specific arguments.

An LLM itself cannot inherently:

- check current weather
- Query your database
- Send an email
- Search live information
- Perform application actions
- Access private APIs

We provide these capabilities as:

Tools

Example:

```
def get_weather(city):  
    ...
```

The LLM can decide:

```
I need current weather.  
  
↓  
call:  
  
get_weather(city="Delhi")
```

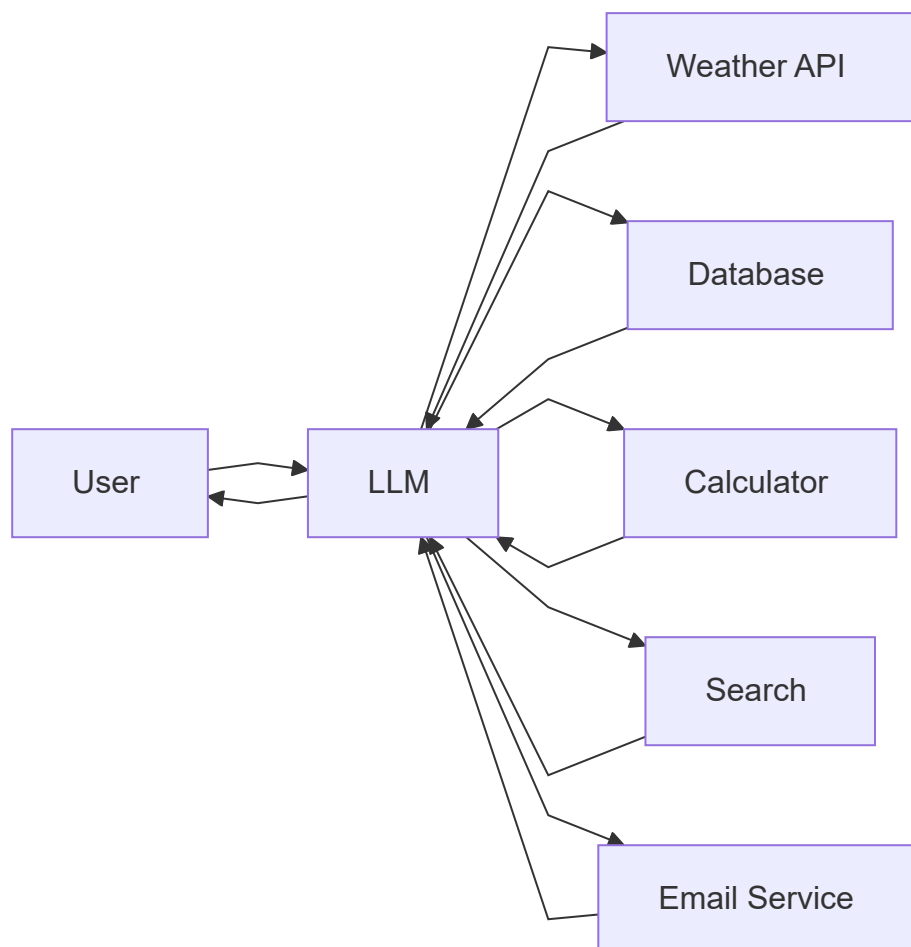
But remember:

The LLM requests the tool call. Your application actually executes it.

2. Why Do Agents Need Tools?

An LLM mainly works with the information available in its context and learned model parameters.

Tools extend its capabilities.



Conceptually:

LLM
+
Tools
=
Reasoning / Decision Making
+
External Capabilities

3. Examples of Tools

Tools can wrap almost any controlled application capability.

API Tool

`get_weather(city)`

Database Tool

`get_user_orders(user_id)`

Search Tool

`search_documents(query)`

Calculator Tool

```
calculate(expression)
```

Email Tool

```
send_email(recipient, subject, message)
```

File Tool

```
read_file(filename)
```

4. Tool Calling vs Normal Function Calling

This distinction is important.

Normal Program

Your code decides which function to execute.

```
if user_wants_weather:  
    get_weather("De1hi")
```

Flow:

Application Logic

↓

Function

↓

Result

LLM Tool Calling

The LLM decides which provided tool is appropriate.

User Request

↓

LLM

↓

Tool Selection

↓

Structured Tool Request

↓

Application Executes Tool

↓

Result Returned to LLM

So the main difference is:

Normal Function Call

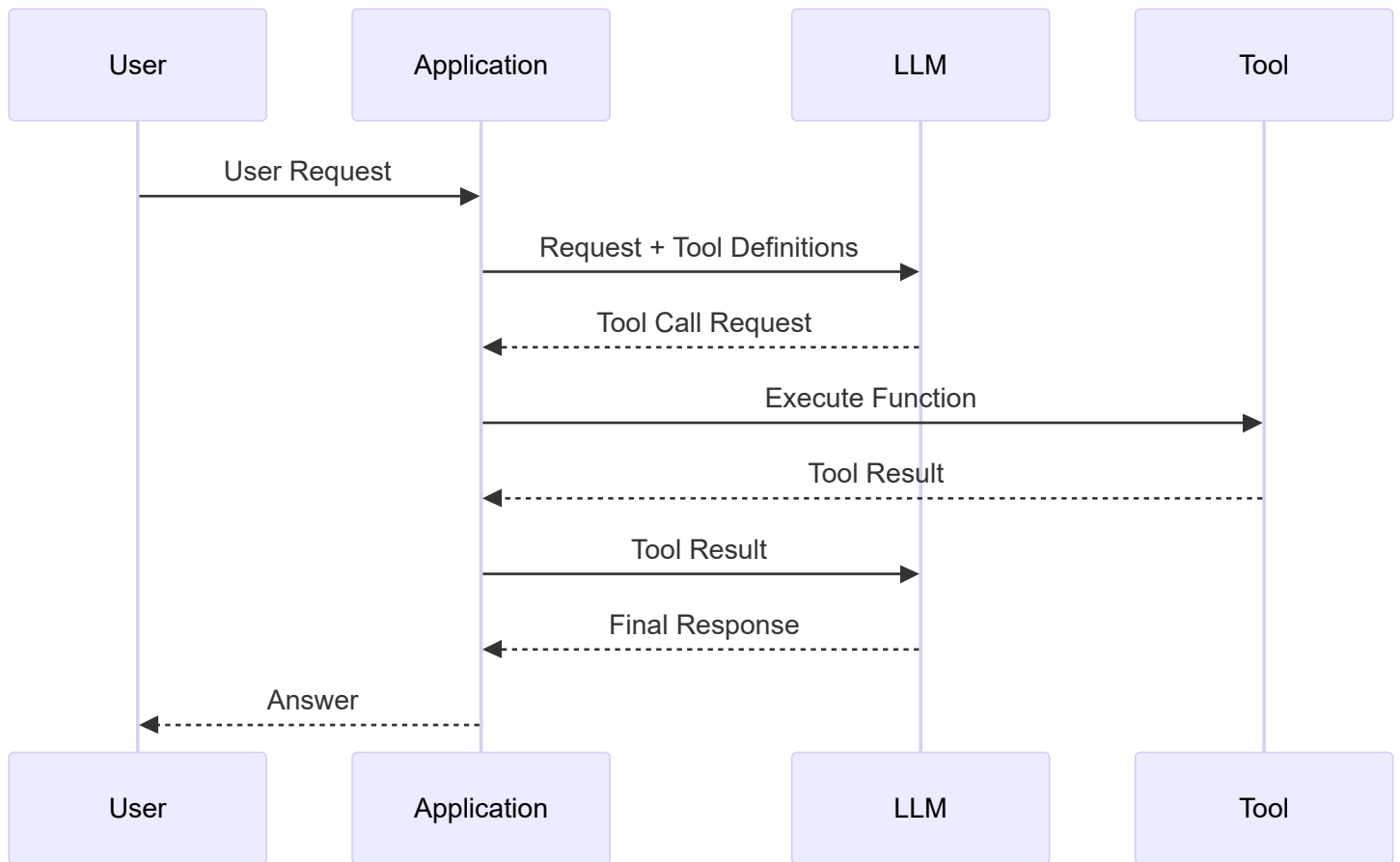
→ Program selects function

Tool Calling

→ LLM selects from provided tools

The actual function is still executed by your application/runtime.

5. Complete Tool-Calling Architecture



This is one of the most important diagrams in Agentic AI.

6. Example

User asks:

what is the weather in Delhi right now?

Available tool:

```
get_weather(city)
```

The LLM determines:

```
Current weather requires external data.
```

It may generate a structured request conceptually like:

```
{  
  "name": "get_weather",  
  "arguments": {  
    "city": "Delhi"  
  }  
}
```

Your application executes:

```
get_weather("Delhi")
```

Suppose the tool returns:

```
{  
  "temperature": 31,  
  "condition": "Rainy"  
}
```

This result is sent back to the LLM.

The LLM then generates:

```
The current temperature in Delhi is 31°C  
with rainy conditions.
```

7. Tool Definition

For an LLM to use a tool correctly, it needs information about that tool.

Typically:

Tool Name

Description

Parameters

Parameter Types

Required Parameters

Example:

Name:

get_weather

Description:

Get current weather information for a city.

Parameters:

city

→ string

→ required

This information forms the:

Tool Schema

8. What is a Tool Schema?



A tool schema is a structured description of a tool that tells the model what the tool does and what arguments it accepts.

Conceptually:

```
{  
  "name": "get_weather",  
  
  "description":  
    "Get current weather for a city",  
  
  "parameters": {  
    "city": "string"  
  }  
}
```

The exact schema format depends on the LLM SDK/framework.

9. Why Tool Schemas Matter

Suppose the tool is only described as:

```
weather()
```

The model does not clearly know:

What does it return?

Which parameters does it require?

When should it be used?

A better definition is:

```
get_current_weather(city)
```

Returns current weather conditions
for the specified city.

city:
Name of the city.

Clear schemas improve:

Tool selection

Argument generation

Reliability

Interpretability

10. Tool Name

Tool names should clearly describe their purpose.

Bad:

```
data()
```

Better:

```
get_weather()
```

Even better when distinction matters:

```
get_current_weather()
```

Another example:

Bad:

```
search()
```

Better:

```
search_product_catalog()
```

II. Tool Description



The description helps the model decide:

```
WHEN should I use this tool?
```

Example:

Use this tool to retrieve current weather information for a specified city.

Compare with:

weather tool.

The first description gives the model more useful information.

12. Tool Parameters

Suppose:

```
def get_weather(city, unit):  
    ...
```

The schema should communicate:

```
city  
→ string  
→ required  
  
unit  
→ string  
→ Celsius or Fahrenheit
```

Conceptually:

```
{  
  "city": "Delhi",  
  "unit": "celsius"  
}
```

13. Required vs Optional Parameters

A tool may contain:

```
Required parameters  
  
Optional parameters
```

Example:

```
search_products(  
  query,  
  max_results=5  
)
```

Here:

```
query  
→ Required  
  
max_results  
→ Optional
```

A good schema should communicate this distinction.

14. JSON Schema Style

Tool APIs commonly use structures based on JSON Schema.

Conceptual example:

```
{
  "type": "object",

  "properties": {

    "city": {
      "type": "string",
      "description": "City name"
    },

    "unit": {
      "type": "string",
      "enum": [
        "celsius",
        "fahrenheit"
      ]
    }
  },

  "required": [
    "city"
  ]
}
```

You do not need to memorize this.

Understand:

Schema

↓

Tells model expected structure

↓

Model generates structured arguments

15. Tool Selection

Suppose an agent has three tools:

```
get_weather(city)
```

```
calculator(expression)
```

```
search_web(query)
```

User asks:

```
What is the weather in Mumbai?
```

The LLM should select:

```
get_weather
```

Not:

```
calculator
```

or:

search_web

when the dedicated weather tool is the appropriate source.

16. How Does the LLM Select a Tool?

The model receives something conceptually like:

User Request:

"What is the weather in Mumbai?"

Available Tools:

1. get_weather
Gets current weather.
2. calculator
Performs mathematical calculations.
3. search_web
Searches the web.

The model determines:

Intent
=
Current Weather

Best Tool
=
get_weather

Then produces the tool call.

17. Tool Calling Does Not Always Mean Tool Usage



Giving tools to an LLM does not mean it should always use them.

Example:

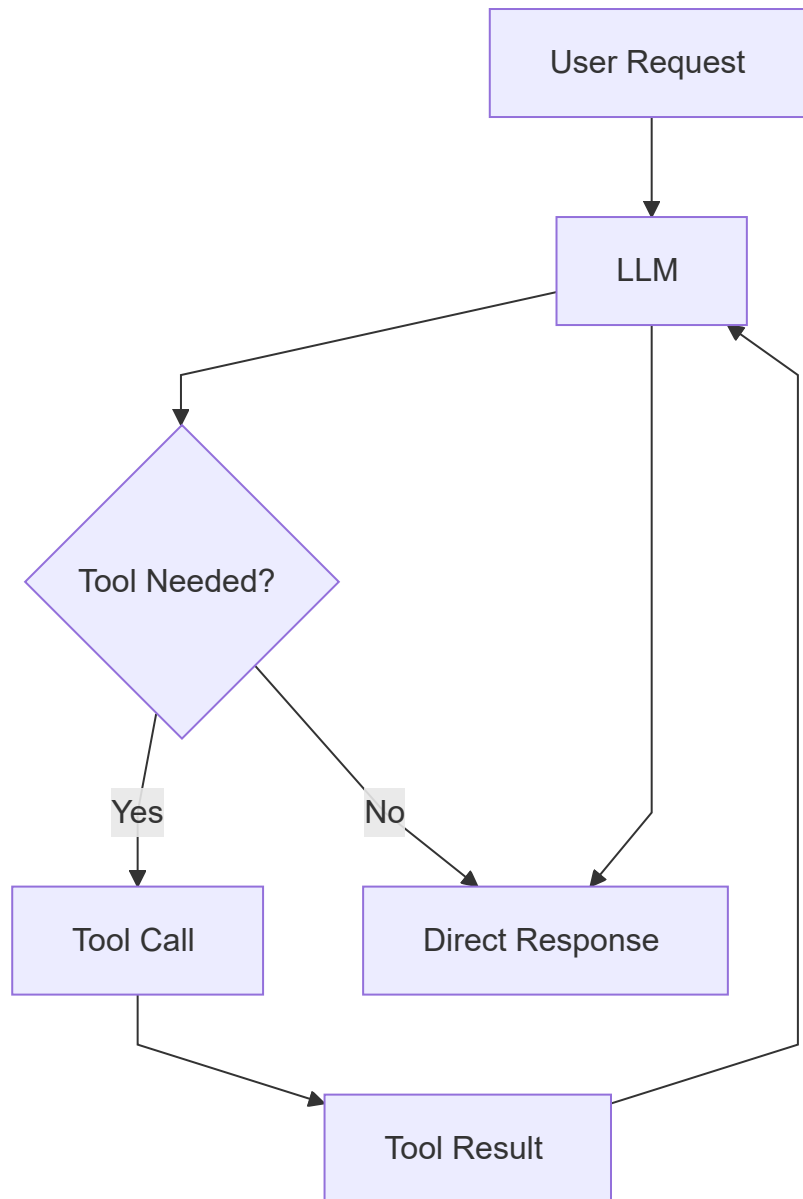
User:
What is photosynthesis?

Available tools:

weather
calculator
search

If no external information is required, the model may simply answer.

Flow:



18. Tool Call Output

When the LLM chooses a tool, its output is not necessarily normal text.

Instead, the API may return structured information containing:

Tool Name

Arguments

Tool Call ID

Conceptually:

```
{  
  "tool_call_id": "call_123",  
  
  "name": "get_weather",  
  
  "arguments": {  
    "city": "Delhi"  
  }  
}
```

The exact representation varies by provider.

19. Tool Call ID

In modern tool-calling systems, each tool request may have an identifier.

Example:

```
call_123
```

Why?

Because the system needs to associate:

Tool Request

with:

Tool Result

This becomes especially useful when multiple tool calls are involved.

20. Tool Execution

Once the application receives:

```
{  
  "name": "get_weather",  
  "arguments": {  
    "city": "Delhi"  
  }  
}
```

the application maps the name to the actual function.

Conceptually:

```
tools = {  
  "get_weather": get_weather,  
  "calculator": calculator  
}
```

Then:

```
tool = tools[tool_name]

result = tool(**arguments)
```

21. Important Security Rule



Never blindly execute arbitrary model-generated commands.

Bad design:

```
eval(model_output)
```

or:

```
LLM generates arbitrary command
↓
Execute directly
```

This can be dangerous.

Instead:

LLM Tool Request



Validate Tool Name



Validate Arguments



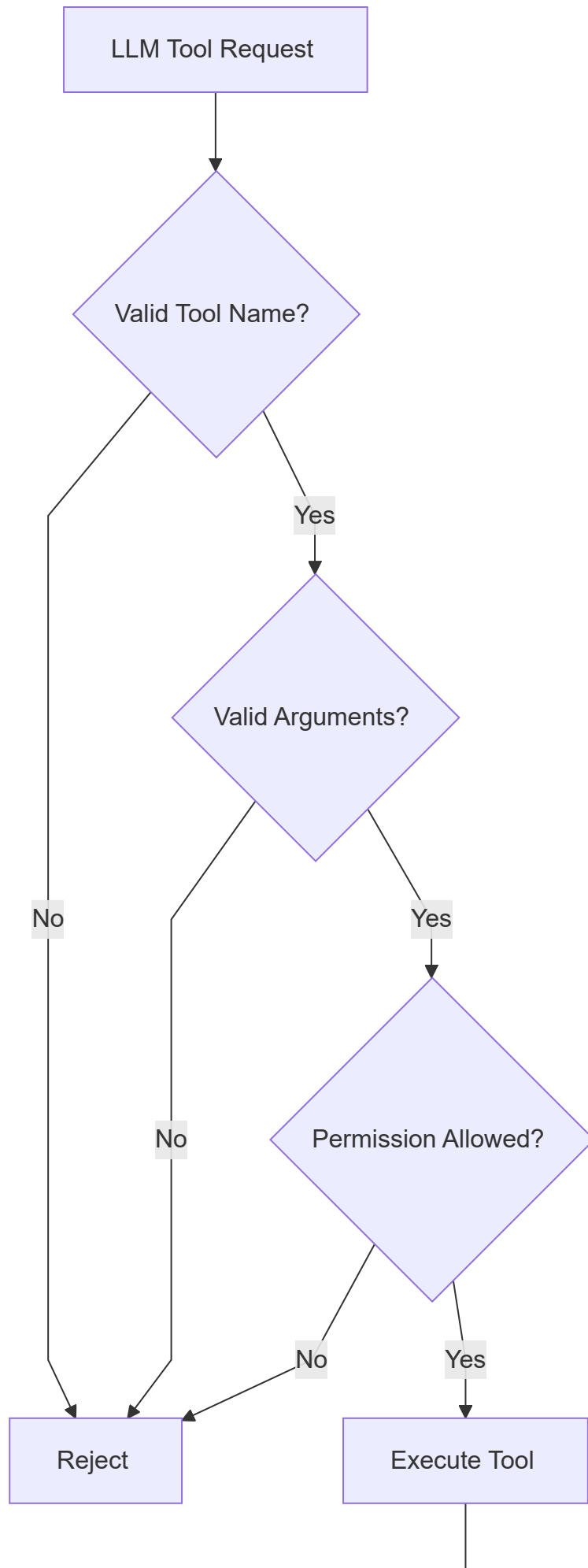
Check Permissions



Execute Approved Function



22. *Safe Tool Execution*



Return Result

This is a better production architecture.

23. Returning Tool Results

Suppose:

```
get_weather("Delhi")
```

returns:

```
{  
  "temperature": 31,  
  "condition": "Rainy"  
}
```

Your application sends this result back to the LLM as a tool result associated with the original call.

Conceptually:

```
Tool:  
get_weather  
  
Result:  
Temperature = 31°C  
Condition = Rainy
```

Now the LLM has the external information required to continue.

24. Why Return Tool Result to the LLM?

Because the model initially knew only:

I need weather information.

After tool execution, it knows:

Weather:
31°C
Rainy

It can now reason over the observation and answer naturally.

Tool Result
↓
LLM
↓
Interpretation
↓
Final Response

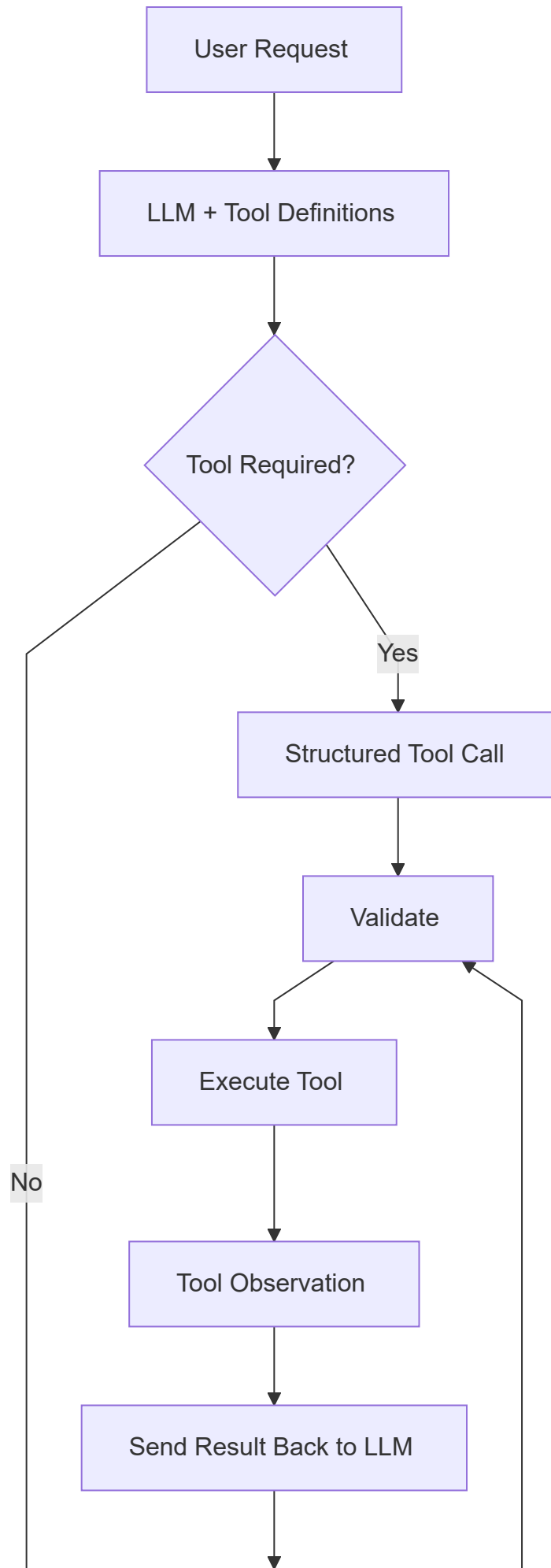
25. Complete Tool-Calling Lifecycle

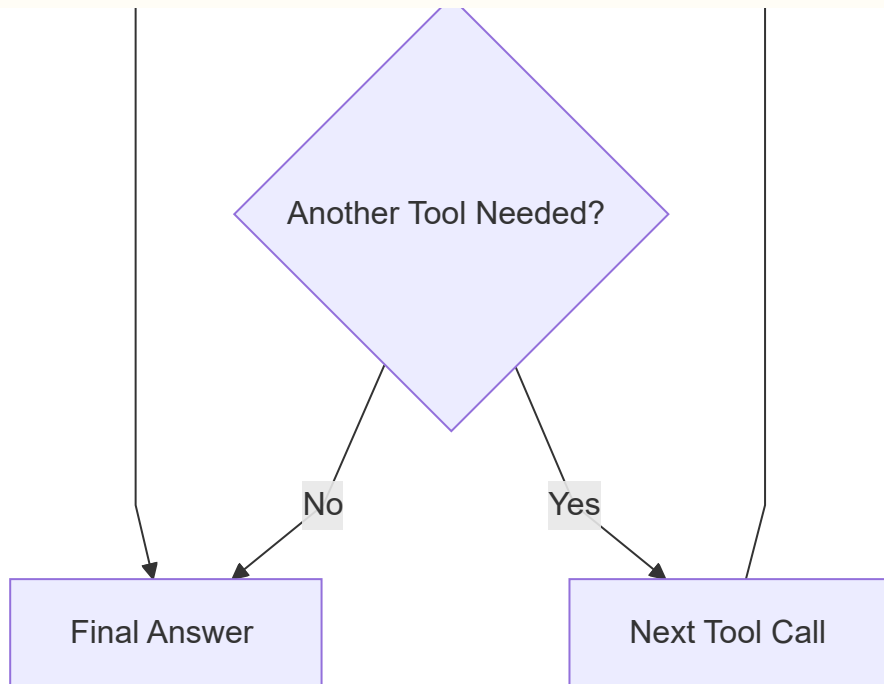
★★★★★ **Must Know**

Remember these steps:

1. User sends request
2. Application sends:
 - User request
 - Tool definitionsto LLM
3. LLM decides whether a tool is required
4. LLM generates structured tool call
5. Application validates request
6. Application executes tool
7. Tool returns result
8. Application sends result back to LLM
9. LLM decides what to do next
10. LLM generates final response

Diagram:





This loop is the foundation of tool-using agents.

26. Tool Calling vs Agent



These are related but not identical.

Tool Calling

User
↓
LLM
↓
Tool
↓
LLM
↓
Answer

It may involve only one tool call.

Agent

```
graph TD; Goal --> LLM1[LLM]; LLM1 --> Tool; Tool --> Observation1[Observation]; Observation1 --> StateUpdate[State Update]; StateUpdate --> LLM2[LLM]; LLM2 --> AnotherTool[Another Tool]; AnotherTool --> Observation2[Observation]; Observation2 --> Dots[...]; Dots --> GoalComplete[Goal Complete];
```

Goal
↓
LLM
↓
Tool
↓
Observation
↓
State Update
↓
LLM
↓
Another Tool
↓
Observation
↓
...
↓
Goal Complete

Therefore:

Tool calling is a capability. An agent is a system that can use capabilities such as tool calling inside a goal-directed control loop.

27. One Tool vs Multiple Tools

An LLM can be given one or multiple tools.

Example:

```
get_weather()  
  
calculator()  
  
search_web()  
  
get_user_profile()  
  
search_documents()
```

Then the model chooses the appropriate one based on the task.

28. Sequential Tool Calling

Sometimes one tool result is required before another tool can be called.

Example:

```
User:  
  
What is 10% of the current temperature  
in Delhi?
```

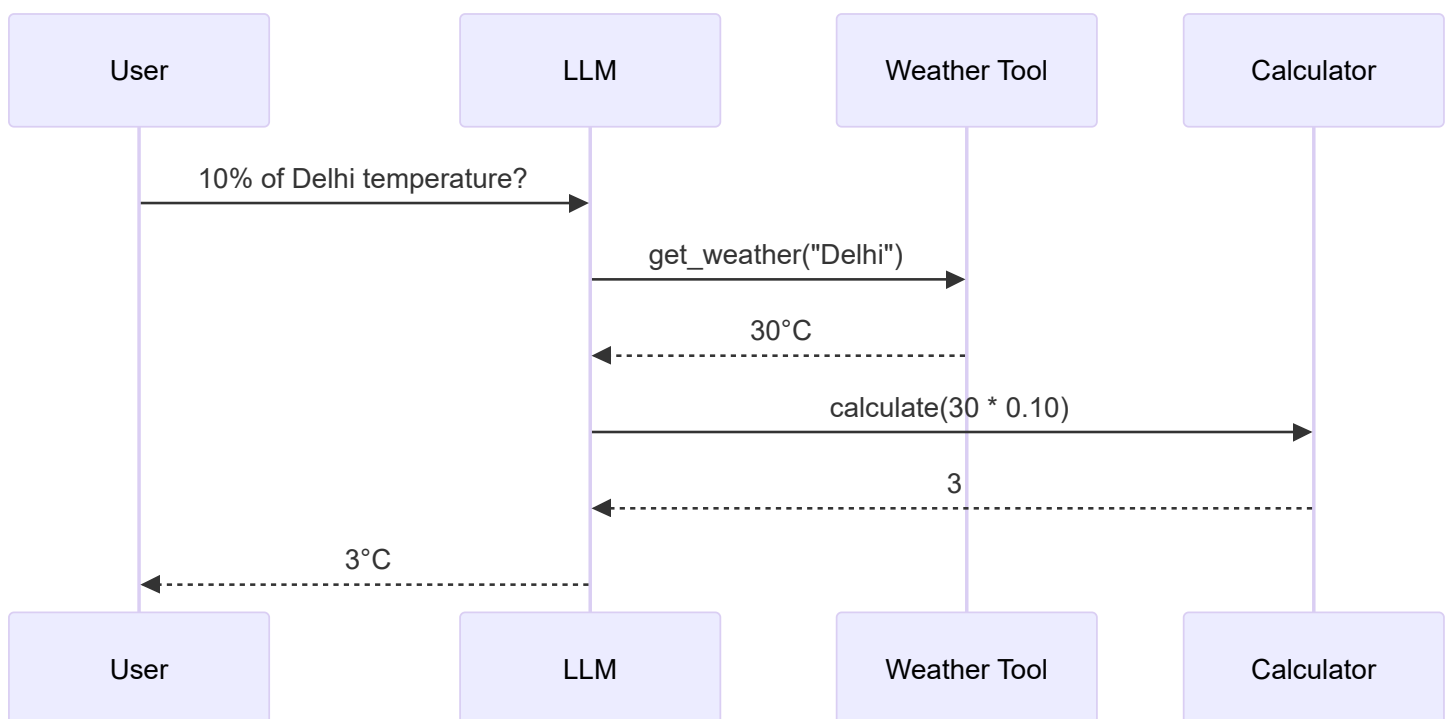
Conceptually:


```
get_weather("Delhi")
↓
Temperature = 30

calculator("30 * 0.10")
↓
3

Final Answer
```

Diagram:



The second action depends on the first observation.

29. Parallel Tool Calling

Sometimes tools are independent.

Example:

Compare the current weather
of Delhi and Mumbai.

Possible calls:

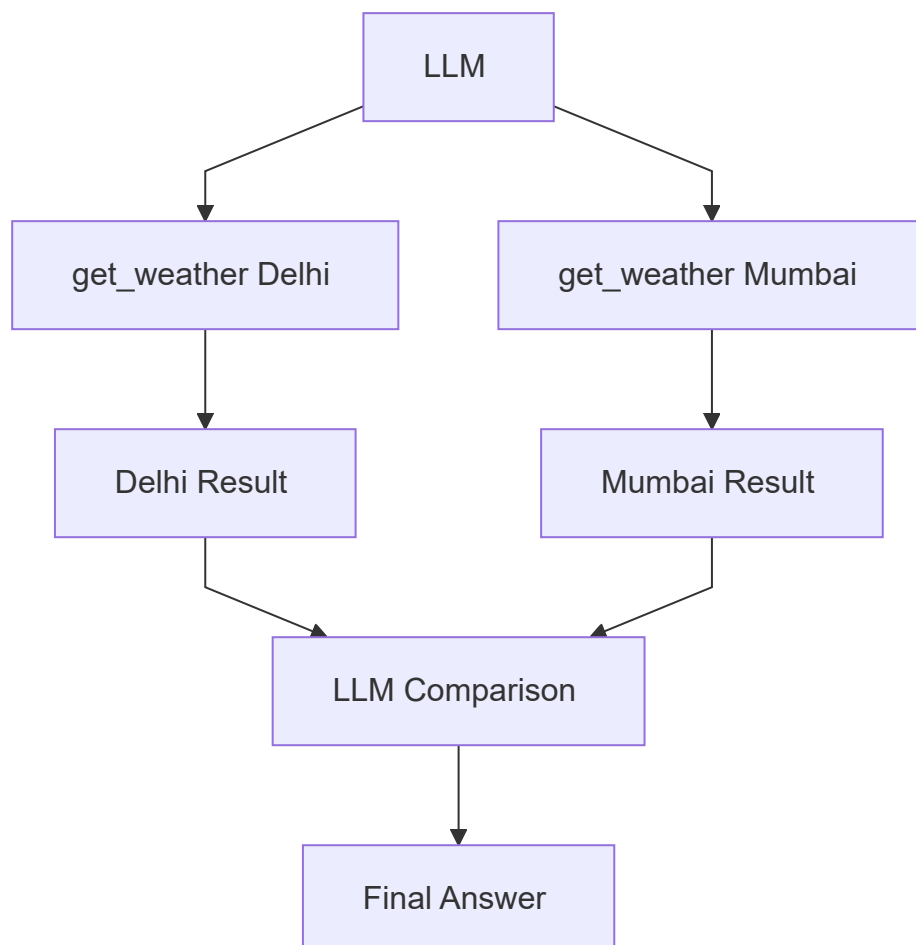
```
get_weather("Delhi")
```

```
get_weather("Mumbai")
```

These calls do not depend on each other.

They may potentially be executed in parallel.

Conceptually:



30. Sequential vs Parallel Tool Calls

Sequential

```
graph TD; A[Tool A] --> B[Result A]; B --> C[Tool B uses Result A]
```

Use when:

```
graph LR; A[B depends on A]
```

Parallel

```
graph TD; L[LLM] --> A[Tool A]; L --> B[Tool B]
```

Use when:

```
graph LR; A[A and B are independent]
```

Parallel execution can reduce latency.

31. Tool Errors



Tools can fail.

Examples:

```
API timeout
```

```
Invalid city
```

```
Database unavailable
```

```
Rate limit
```

```
Incorrect arguments
```

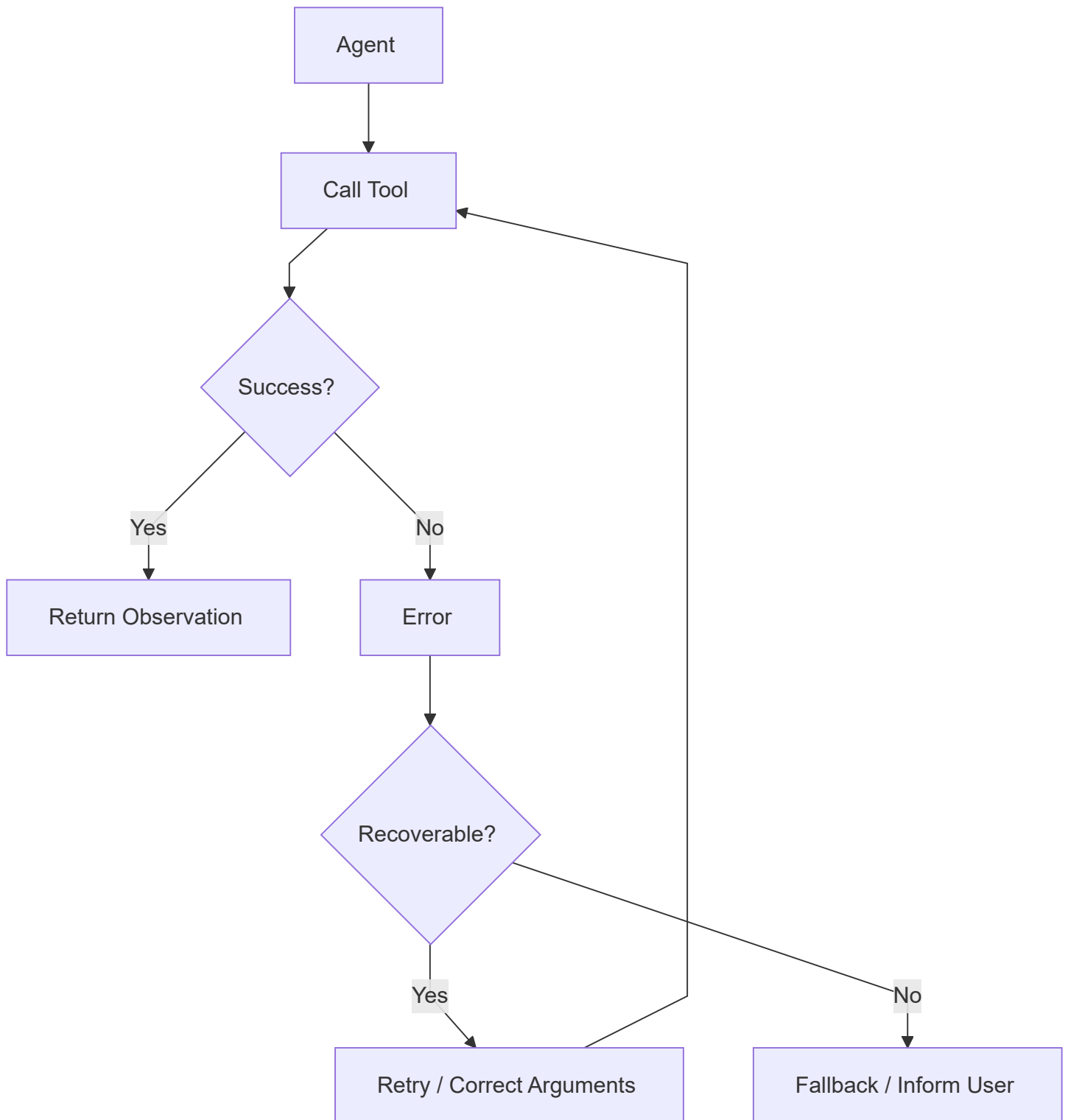
```
Authentication failure
```

Example result:

```
{  
  "error": "City not found"  
}
```

The agent should not blindly treat this as valid weather data.

32. Tool Error Flow



33. Tool Validation

Before execution, validate arguments.

Example:

```
def get_weather(city):  
  
    if not isinstance(city, str):  
        raise ValueError(  
            "city must be a string"  
        )  
  
    ...
```

More robust applications may use schema validation libraries.

The principle is:

```
Never trust model-generated arguments blindly.
```

34. Read Tools vs Write Tools

This is an important practical distinction.

Read Tool

Retrieves information.

Examples:

```
get_weather()

search_database()

read_document()
```

Usually lower risk.

Write / Action Tool

Changes something.

Examples:

```
send_email()

delete_record()

make_payment()

create_booking()

update_database()
```

These can have real-world consequences.

35. Sensitive Tool Architecture

For high-impact actions:

Agent



Proposes Action



Validate

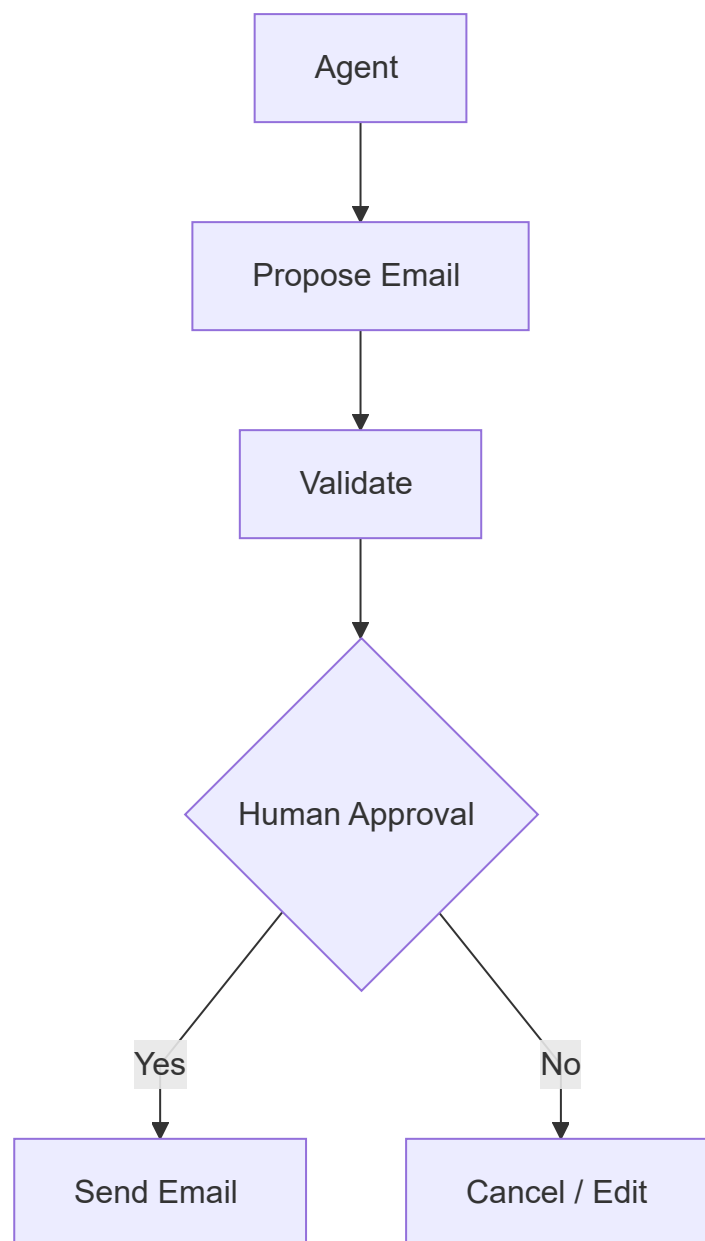


Human Approval



Execute

Example:



Do not give unnecessary unrestricted access to action tools.

36. Tool Permissions



An agent should only receive tools it is allowed to use.

Example:

Customer Support Agent

Allowed:

- ✓ search_orders
- ✓ check_delivery_status

Not Allowed:

- X delete_user
- X modify_payment

This follows the security principle of:

Least Privilege

Give the agent only the capabilities necessary for its task.

37. Tool Calling Implementation

Now let's implement a basic LLM tool-calling flow.

The exact SDK syntax differs between providers.

Therefore, first understand the generic architecture.

We will use Python-style pseudocode close to modern LLM APIs.

38. Step 1 – Create Tool

```
def get_weather(city):  
  
    weather_data = {  
        "Delhi": {  
            "temperature": 31,  
            "condition": "Rainy"  
        },  
  
        "Mumbai": {  
            "temperature": 29,  
            "condition": "Cloudy"  
        }  
    }  
  
    return weather_data.get(  
        city,  
        {  
            "error": "City not found"  
        }  
    )
```

39. Step 2 – Define Tool Schema

Conceptually:

```
weather_tool = {  
    "name": "get_weather",  
  
    "description":  
        "Get the current weather for a city.",  
  
    "parameters": {  
  
        "type": "object",  
  
        "properties": {  
  
            "city": {  
                "type": "string",  
                "description":  
                    "Name of the city"  
            }  
        },  
  
        "required": [  
            "city"  
        ]  
    }  
}
```

The model now knows:

```
Tool:  
get_weather  
  
Input:  
city
```

40. Step 3 – Send Request to LLM

Conceptually:

```
response = llm.generate(  
    messages=[  
        {  
            "role": "user",  
            "content":  
                "What is the weather in Delhi?"  
        }  
    ],  
  
    tools=[  
        weather_tool  
    ]  
)
```

The model may return:

```
Tool Call:  
get_weather  
  
Arguments:  
city = Delhi
```

41. Step 4 – Read Tool Request

Conceptually:

```
tool_call = response.tool_call

tool_name = tool_call.name

arguments = tool_call.arguments
```

Result:

```
tool_name = "get_weather"

arguments = {
    "city": "Delhi"
}
```

42. Step 5 – Validate Tool

```
available_tools = {
    "get_weather": get_weather
}
```

Check:

```
if tool_name not in available_tools:
    raise ValueError(
        "Unknown tool"
    )
```

Then validate arguments.

43. Step 6 – Execute Tool

```
tool_function = available_tools[
    tool_name
]

result = tool_function(
    **arguments
)
```

Result:

```
{
    "temperature": 31,
    "condition": "Rainy"
}
```

44. Step 7 – Send Tool Result Back

Conceptually:

```
messages.append(  
    tool_call_message  
)  
  
messages.append({  
    "role": "tool",  
  
    "tool_call_id":  
        tool_call.id,  
  
    "content":  
        str(result)  
})
```

Then call the LLM again:

```
final_response = llm.generate(  
    messages=messages,  
    tools=[weather_tool]  
)
```

45. Step 8 – Final Response

The model now sees:

User:
What is the weather in Delhi?

Tool Request:
get_weather("Delhi")

Tool Result:
31°C, Rainy

It can generate:

The current weather in Delhi is rainy,
with a temperature of around 31°C.

46. Complete Generic Implementation



```
# Tool
def get_weather(city):

    return {
        "city": city,
        "temperature": 31,
        "condition": "Rainy"
    }

# Available functions
available_tools = {
    "get_weather": get_weather
}

# Tool schema
tools = [
    {
        "name": "get_weather",

        "description":
            "Get current weather for a city.",

        "parameters": {

            "type": "object",
```



```
        "properties": {
            "city": {
                "type": "string"
            }
        },
        "required": [
            "city"
        ]
    }
}
```

```
# User conversation
```

```
messages = [
    {
        "role": "user",
        "content":
            "What is the weather in Delhi?"
    }
]
```

```
# First LLM call
```

```
response = llm.generate(
    messages=messages,
    tools=tools
)
```

```
# Check tool request
```

```
if response.has_tool_call:

    tool_call = response.tool_call

    tool_name = tool_call.name

    arguments = tool_call.arguments
```

```
# Validate tool
if tool_name not in available_tools:

    raise ValueError(
        "Invalid tool"
    )
```

```
# Execute tool
result = available_tools[
    tool_name
](**arguments)
```

```
# Preserve model tool request
messages.append(
    response.message
)
```

```
# Add tool observation
messages.append({
    "role": "tool",

    "tool_call_id":
        tool_call.id,

    "content":
        str(result)
})
```

```
# Second LLM call
final_response = llm.generate(
    messages=messages,
    tools=tools
)
```

```
print(  
    final_response.text  
)
```

Again:

Do not memorize provider-specific syntax. Understand the flow.

47. Why Two LLM Calls?



A common beginner question:

```
why call the LLM twice?
```

First LLM Call

Purpose:

```
Understand request  
+  
Decide which tool to use  
+  
Generate arguments
```

Tool Execution

```
Application executes tool
```

Second LLM Call

Purpose:

```
Interpret tool result
+
Generate user-friendly answer
+
Possibly decide another tool is needed
```

So:

```
LLM Call 1
↓
Tool
↓
LLM Call 2
```

48. From Tool Calling to Agent Loop

What if the second LLM call requests another tool?

Instead of writing:

```
LLM
→ Tool
→ LLM
```

only once, create a loop.

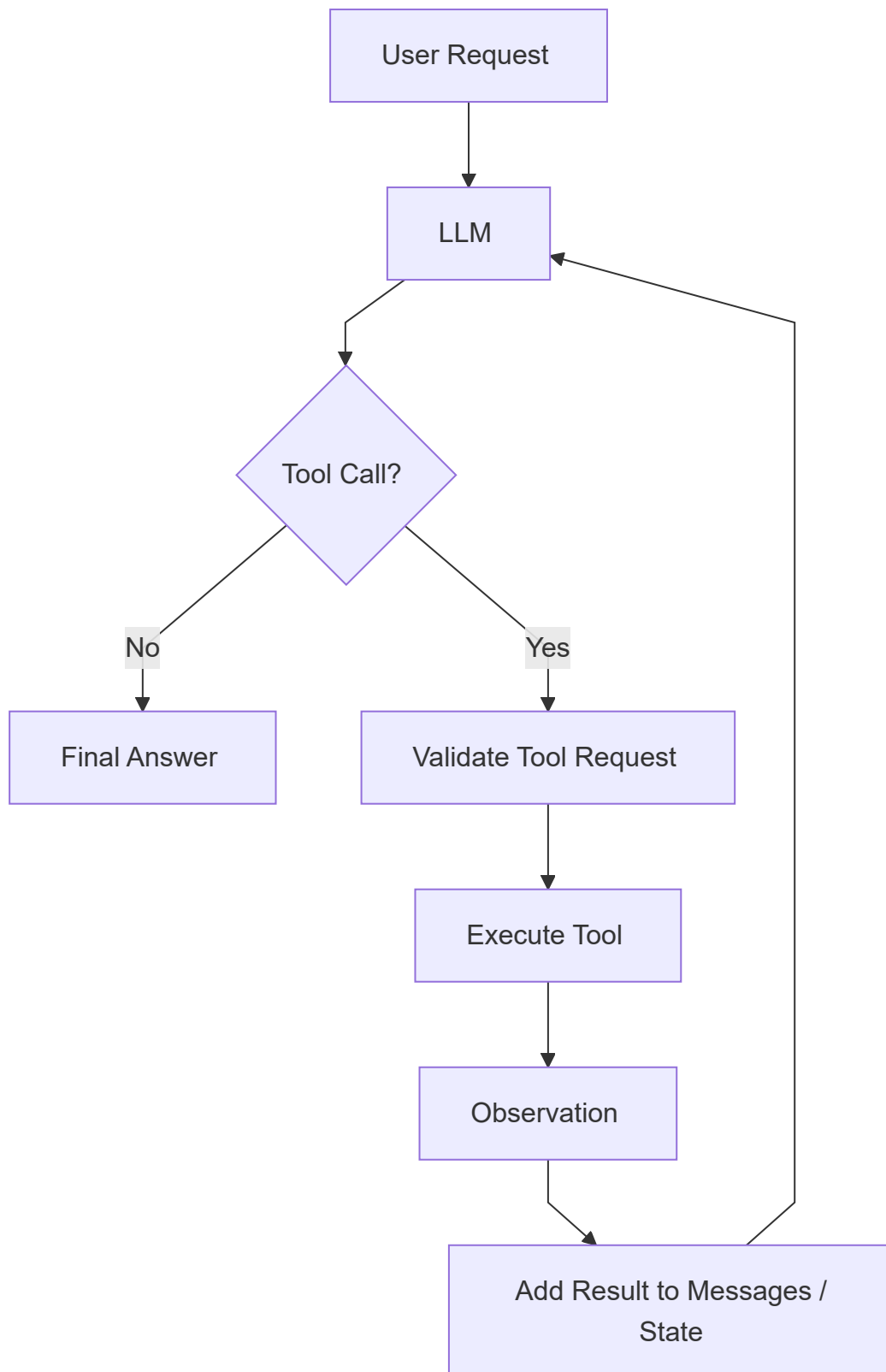
```
while True:
```

```
response = llm.generate(  
    messages=messages,  
    tools=tools  
)  
  
if not response.has_tool_call:  
  
    return response.text  
  
tool_call = response.tool_call  
  
result = execute_tool(  
    tool_call  
)  
  
messages.append(  
    response.message  
)  
  
messages.append(  
    create_tool_result(  
        tool_call,  
        result  
    )  
)
```

Now we have the foundation of a:

Tool-Using Agent

49. Tool-Calling Agent Loop



This should look familiar.

It is essentially the agent loop from Chapter 1.

50. Tool Calling + State

As tools execute, results become part of state.

Example:

```
state = {  
  "messages": [...],  
  
  "tool_results": [  
    {  
      "tool": "get_weather",  
      "result": {  
        "temperature": 31  
      }  
    }  
  ],  
  
  "steps": 1  
}
```

Then the next decision uses the updated state.

51. Tool Calling + Memory

Suppose a user previously told the system:

```
My preferred city is Delhi.
```

Memory may contain:

```
preferred_city = Delhi
```

Later:

```
User:  
What's the weather there?
```

The system can combine:

```
User Request  
+  
Memory  
+  
Weather Tool
```

to determine:

```
get_weather("Delhi")
```

We will study memory properly in Chapter 3.

52. Tool Calling + RAG

RAG can itself be exposed as a tool.

Example:

```
search_knowledge_base(query)
```


Agent:

```
User Question
↓
Agent
↓
Need Internal Knowledge?
↓
RAG Tool
↓
Relevant Documents
↓
Agent
↓
Answer
```

This is one way RAG becomes part of a larger agentic system.

53. API Tool

A real tool often wraps an API.

```
def get_weather(city):

    response = weather_api.get(
        city=city
    )

    return response
```

The LLM does not need to know:

API URL

Authentication details

HTTP implementation

Secret API key

It only needs the safe interface:

```
get_weather(city)
```

54. Database Tool

Example:

```
def get_order_status(order_id):  
  
    order = database.find_order(  
        order_id  
    )  
  
    return {  
        "status": order.status  
    }
```

Tool schema:

```
get_order_status(order_id)
```

Use this tool to retrieve
the status of an existing order.

The model should not receive unrestricted database access if a narrow function is sufficient.

55. Good Tool Design



A good tool should have:

Clear Purpose

Clear Name

Clear Description

Typed Parameters

Validated Inputs

Structured Output

Controlled Permissions

Useful Error Messages

Prefer:

```
get_order_status(order_id)
```

over a dangerous generic tool such as:

```
execute_database_query(query)
```

when the narrower tool can solve the task.

56. Tool Granularity

Tools should not be:

Too Broad

or unnecessarily:

Too Tiny

Example of too broad:

```
do_everything(command)
```

Problems:

Hard to control

Hard to validate

Security risk

Hard for model to select correctly

Better:

```
get_weather(city)
```

```
search_orders(user_id)
```

```
create_support_ticket(issue)
```

Each tool has a clear responsibility.

57. Common Tool-Calling Mistakes

Mistake 1 – Thinking the LLM Executes the Function

Remember:

```
LLM
```

```
→ Requests
```

```
Application
```

```
→ Executes
```

Mistake 2 – Poor Tool Description

Unclear descriptions lead to poor selection.

Mistake 3 – No Argument Validation

Never blindly trust model-generated arguments.

Mistake 4 – Giving Too Many Tools

Unnecessary tools increase complexity.

Mistake 5 – Giving Dangerous Generic Tools

Avoid unrestricted:

Shell execution

Database queries

File deletion

unless absolutely necessary and strongly controlled.

Mistake 6 – No Error Handling

Tools can fail.

Design for failure.

Mistake 7 – No Tool Result Returned to Model

The model needs the observation to continue reasoning.

Mistake 8 – Using Tools When Not Required

Not every question requires an external action.

58. Important Interview Questions

Q1. What is tool calling? ★ ★ ★ ★ ★

Answer:

Tool calling is a mechanism where an LLM selects an externally provided function or capability and generates structured arguments for it. The application executes the tool and returns the result to the model.

Q2. Does the LLM execute the tool directly?

★ ★ ★ ★ ★

Answer:

No. The LLM normally generates a structured tool-call request. The application or agent runtime validates the request, executes the actual function and sends the result back to the model.

Q3. What is a tool schema? ★ ★ ★ ★ ★

Answer:

A tool schema is a structured definition describing a tool's name, purpose, parameters, parameter types and required inputs so that the LLM knows when and how to call it.

Q4. Why are tool descriptions important?

Answer:

Tool descriptions help the LLM understand what each tool does and determine which tool is appropriate for the user's request.

Q5. Normal function call vs LLM tool call?



Answer:

In a normal function call, application logic explicitly determines which function to execute. In LLM tool calling, the model selects an appropriate tool from the provided definitions and generates the required arguments.

Q6. What happens after an LLM generates a tool call? ★ ★ ★ ★ ★

Answer:

The application validates the requested tool and arguments, executes the corresponding function, obtains the result and sends that result back to the LLM for further processing.

Q7. Why might tool calling require multiple LLM calls?

Answer:

The first LLM call may determine which tool to use. After the tool executes, another LLM call is needed to interpret the result, generate the final answer or decide whether another tool is required.

Q8. Sequential vs parallel tool calling?



Answer:

Sequential tool calling is used when a later tool depends on the result of an earlier tool. Parallel tool calling can be used when multiple tool calls are independent and can execute simultaneously.

Q9. Tool calling vs AI agent? ★ ★ ★ ★ ★

Answer:

Tool calling is a capability that allows an LLM to request external functions. An AI agent is a broader goal-directed system that may repeatedly use tool calling, state, memory and control logic until a task is completed.

Q10. How should tool errors be handled?

Answer:

Tool failures should be represented clearly and handled through controlled retries, corrected arguments, alternative actions or graceful failure rather than allowing unlimited retries.

Q11. What is the principle of least privilege?

Answer:

The principle of least privilege means giving an agent only the tools and permissions necessary to perform its intended task, reducing security and misuse risks.

Q12. Why shouldn't you execute arbitrary model output?

Answer:

Model output is untrusted input. Arbitrarily executing it can cause security vulnerabilities or unintended actions. Tool names, arguments and permissions should always be validated before execution.

Q13. Can RAG be used as an agent tool?

Answer:

Yes. A retrieval system can be exposed as a tool such as

`search_knowledge_base()`, allowing an agent to retrieve relevant documents when needed.

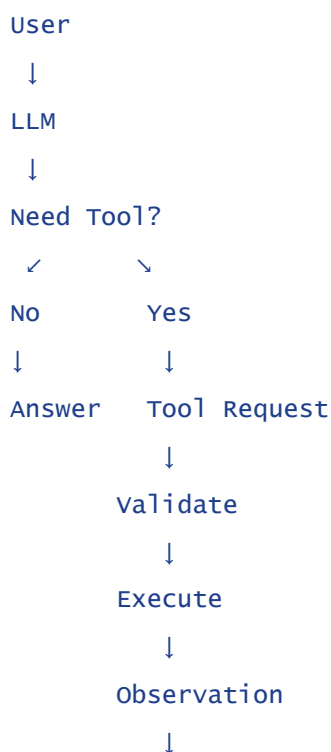
Q14. What makes a good tool?

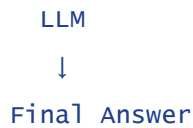
Answer:

A good tool has a clear purpose, descriptive name, well-defined typed parameters, validated inputs, structured outputs, controlled permissions and predictable error handling.

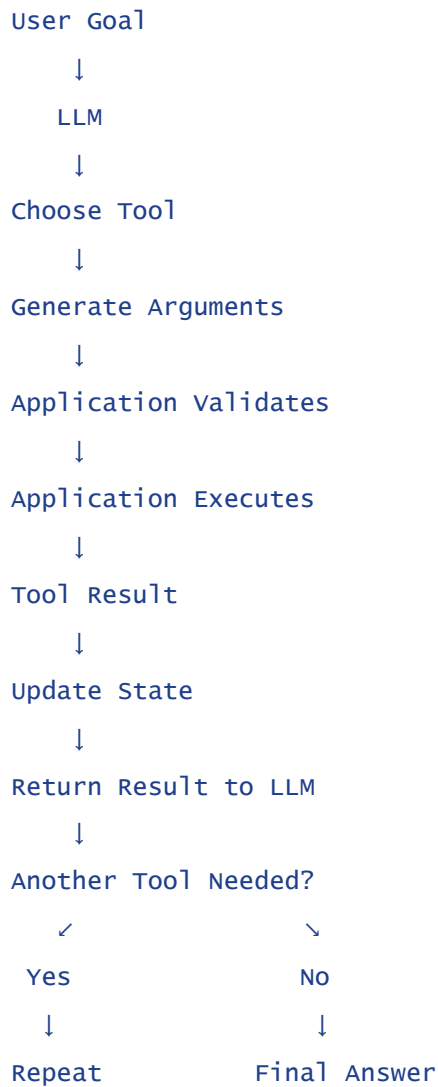
59. Final Mental Model

Remember the basic tool-calling flow:

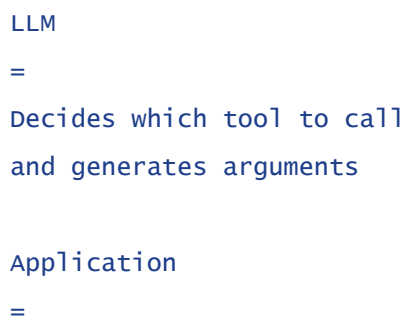




And the complete agent version:



The most important distinction:



Validates and executes tool

Tool

=

Performs external capability

Observation

=

Result returned to agent

And finally:

Tool Calling

+

Loop

+

State

+

Goal

=

Foundation of an AI Agent

Tool calling bridges the gap between an LLM's reasoning capabilities and real-world application capabilities by allowing the model to request controlled external actions while the application remains responsible for validation and execution.

Agentic AI Mastery Handbook

Chapter 2 – Tool Calling

Part 2 – Multiple Tools, Real Implementation & Tool-Calling Agent

Interview Importance: ★ ★ ★ ★ ★

Difficulty: Intermediate

Prerequisites: Chapter 2 – Part 1

1. From Single Tool to Tool-Calling Agent

In Part 1, we learned the basic flow:

```
graph TD
    User --> LLM1[LLM]
    LLM1 --> ToolCall[Tool Call]
    ToolCall --> AppExec[Application Executes Tool]
    AppExec --> ToolResult[Tool Result]
    ToolResult --> LLM2[LLM]
    LLM2 --> FinalAnswer[Final Answer]
```

Real agents usually have access to multiple tools.

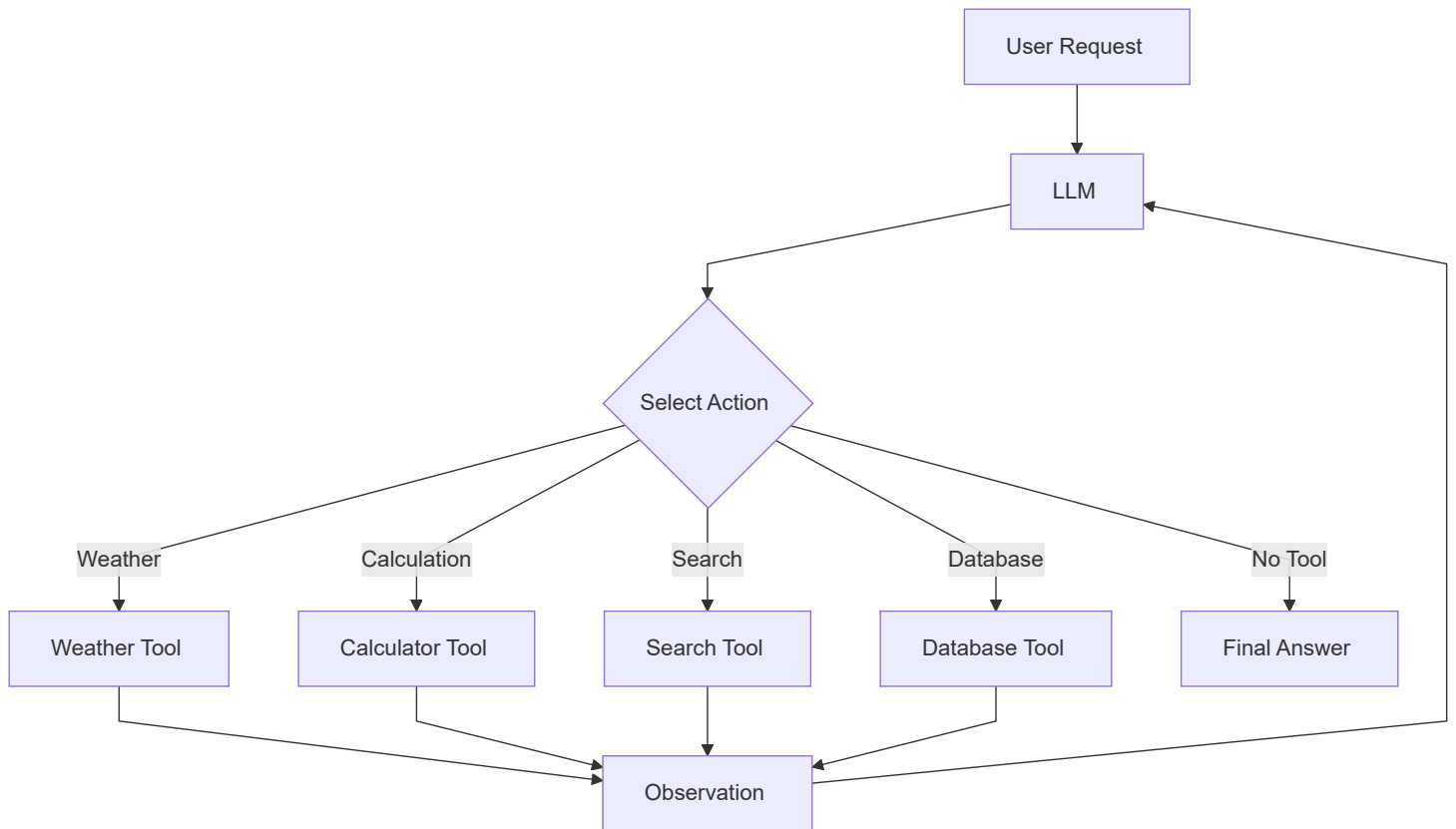
Example:

```
graph TD
    subgraph Tools
        direction TB
        W[Weather Tool]
        C[Calculator Tool]
        S[Search Tool]
        D[Database Tool]
    end
```

Now the LLM must decide:

```
graph TD
    Q1[Do I need a tool?]
    Q2[which tool?]
    Q3[what arguments?]
    Q4[Do I need another tool afterward?]
    Q5[Can I answer now?]
```

2. Multiple Tool Architecture



The LLM acts as the:

Decision Layer

while tools provide:

External Capabilities

3. Example Tools

Let's create three simple tools.

```
def get_weather(city):  
  
    data = {  
        "delhi": {  
            "temperature": 32,  
            "condition": "rainy"  
        },  
  
        "mumbai": {  
            "temperature": 29,  
            "condition": "cloudy"  
        }  
    }  
  
    return data.get(  
        city.lower(),  
        {"error": "City not found"}  
    )
```

Calculator:

```
def calculator(a, b, operation):  
  
    if operation == "add":  
        return a + b  
  
    if operation == "subtract":  
        return a - b  
  
    if operation == "multiply":  
        return a * b  
  
    if operation == "divide":
```

```
        if b == 0:
            return {
                "error": "Division by zero"
            }

        return a / b

    return {
        "error": "Unsupported operation"
    }
```

Search-style tool:

```
def search_knowledge_base(query):

    return {
        "query": query,
        "result":
            "Relevant information found."
    }
```

4. Tool Registry



Instead of manually checking every possible function:

```
if tool_name == "weather":
    ...

elif tool_name == "calculator":
    ...
```

we can maintain a tool registry.

```
available_tools = {  
  
    "get_weather":  
        get_weather,  
  
    "calculator":  
        calculator,  
  
    "search_knowledge_base":  
        search_knowledge_base  
}
```

Now execution becomes:

```
tool = available_tools[  
    tool_name  
]  
  
result = tool(  
    **arguments  
)
```

5. Why Tool Registry?

A tool registry gives us one central mapping:

```
Tool Name  
↓  
Actual Function
```

Example:

```
"get_weather"  
  ↓  
get_weather()
```

Benefits:

```
Cleaner code  
  
Easy tool lookup  
  
Easy validation  
  
Easy addition/removal of tools
```

6. Multiple Tool Schemas

The LLM also needs schemas for every available tool.

Conceptually:

```
tools = [  
  {  
    "name": "get_weather",  
  
    "description":  
      "Get current weather for a city.",  
  
    "parameters": {  
      "city": "string"  
    }  
  },  
  
  {  
    "name": "calculator",
```

```
[{"description": "Perform a mathematical operation.", "parameters": {"a": "number", "b": "number", "operation": "string"}}, {"name": "search_knowledge_base", "description": "Search the internal knowledge base.", "parameters": {"query": "string"}}]
```

The exact API representation depends on the provider.

7. Tool Selection Example

Available tools:

```
get_weather  
  
calculator  
  
search_knowledge_base
```

User:

what is the weather in Delhi?

Expected:

get_weather

User:

calculate 35×17 .

Expected:

calculator

User:

search our knowledge base for wheat irrigation.

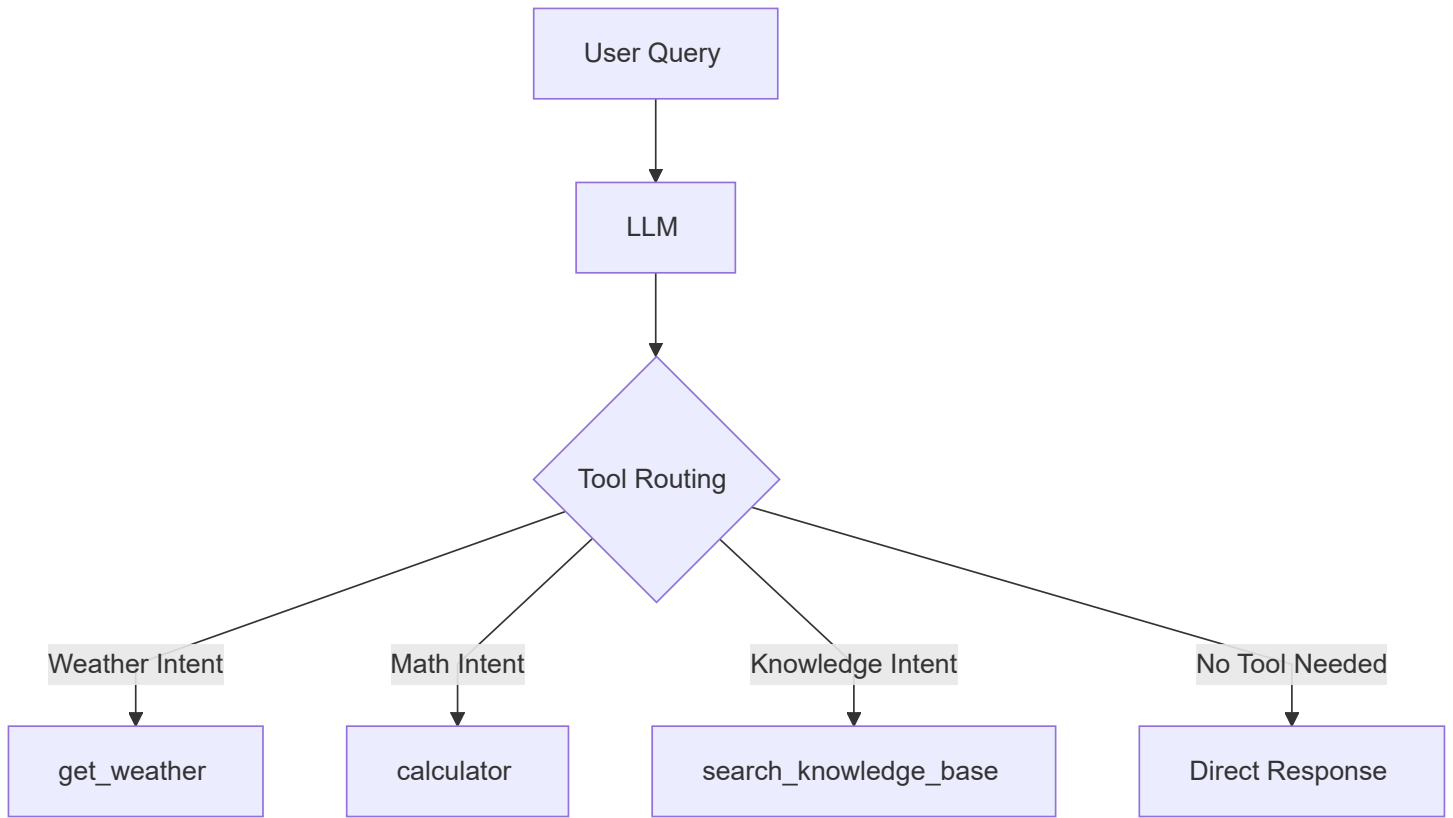
Expected:

search_knowledge_base

8. Tool Routing

The process of choosing the appropriate tool can be thought of as:

Tool Routing



In an LLM tool-calling system, the model can perform this routing dynamically.

9. Sequential Tool Calling



Suppose:

User:

what is double the current temperature
in Delhi?

The agent needs:

Step 1

→ Current Delhi temperature

Step 2

→ Multiply temperature by 2

Therefore:

```
get_weather("Delhi")
```

↓

```
temperature = 32
```

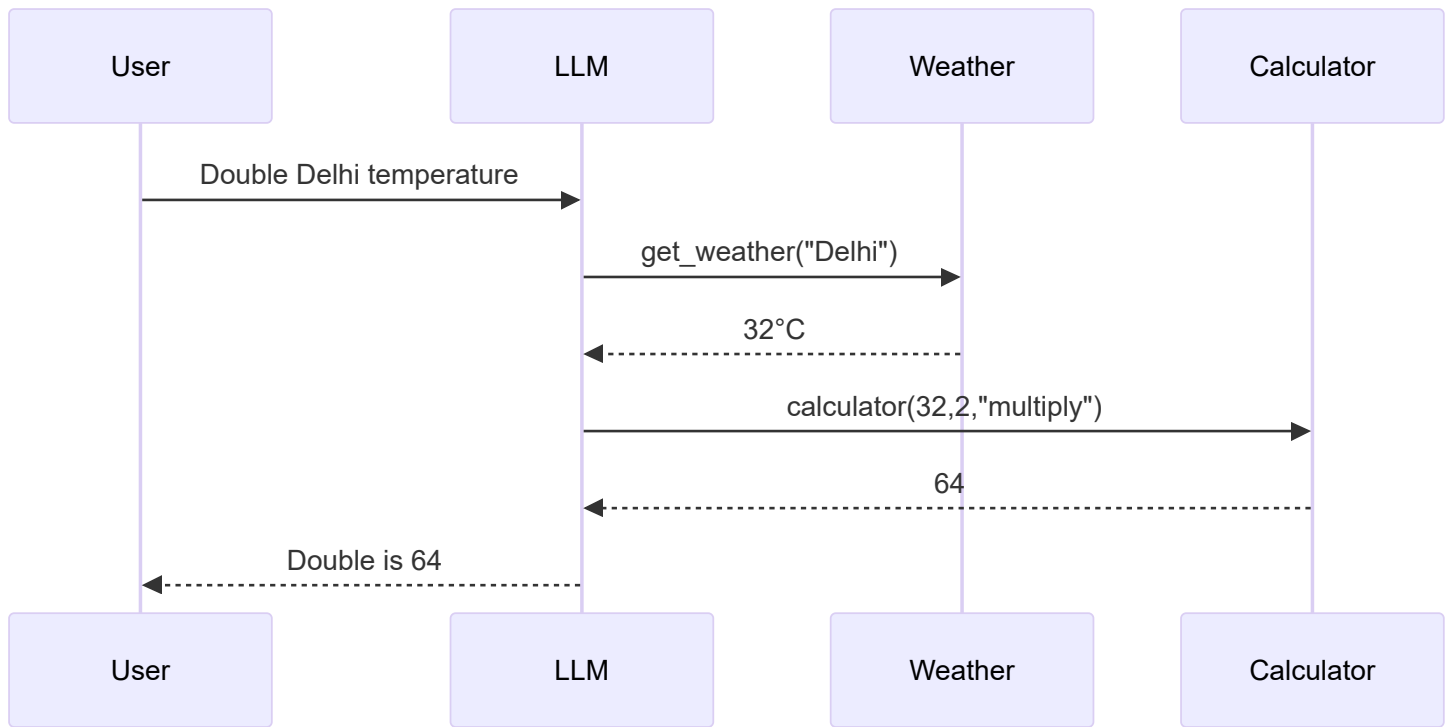
↓

```
calculator(32, 2, "multiply")
```

↓

```
64
```

10. Sequential Execution Flow



The important point:

Tool 2 depends on Tool 1 result.

Therefore they must execute sequentially.

11. Parallel Tool Calling



Suppose:

Compare Delhi and Mumbai weather.

We need:

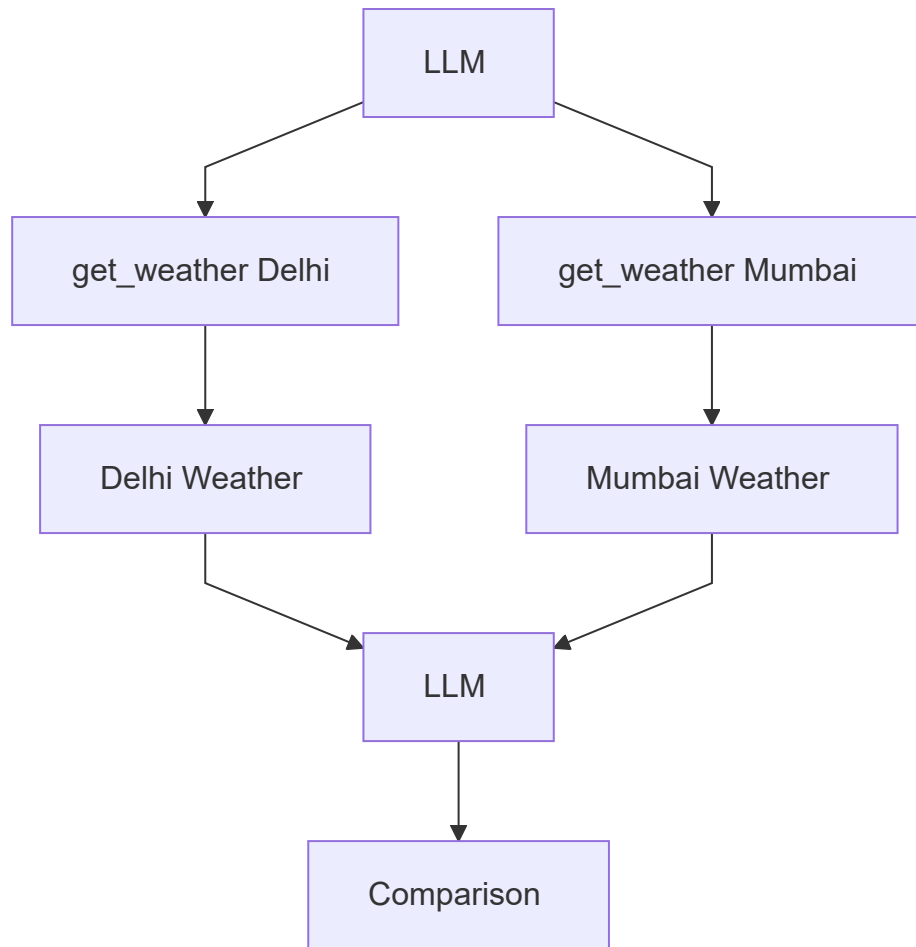
```
get_weather("Delhi")  
  
get_weather("Mumbai")
```

Neither call depends on the other.

Therefore:



Diagram:



12. Why Parallel Tool Calls Matter

Suppose:

weather Tool
=
2 seconds

Two sequential calls may take roughly:

2 + 2
=
4 seconds

Two independent parallel calls may finish closer to:

~2 seconds

depending on infrastructure and overhead.

Therefore parallel calls can reduce:

Latency

13. Tool Dependency

Before parallelizing tools, ask:

Does Tool B require Tool A's result?

If:

*YES
→ Sequential*

If:

*NO
→ Potentially Parallel*

Example:

```
Current Bitcoin price
↓
Calculate 10% of price
```

Sequential.

But:

```
Delhi weather

Mumbai weather
```

Independent.

14. Tool Result as Observation

Each tool result becomes an:

Observation

Example:

```
Action:

get_weather("Delhi")
```

Observation:

```
{  
  "temperature": 32,  
  "condition": "rainy"  
}
```

The agent can now make its next decision using this observation.

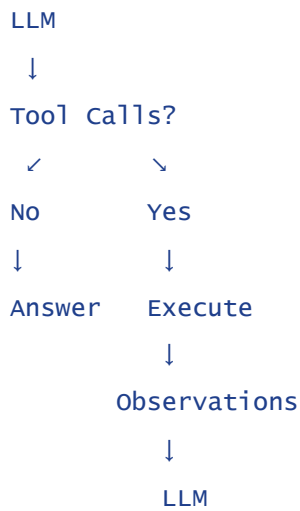
15. Multiple Tool Agent Loop



The basic loop becomes:

```
while True:  
  
    response = call_llm(  
        messages,  
        tools  
    )  
  
    if response.has_no_tool_calls():  
  
        return response.text  
  
    for tool_call in response.tool_calls:  
  
        result = execute_tool(  
            tool_call  
        )  
  
        add_tool_result(  
            messages,  
            tool_call,  
            result  
        )
```

Conceptually:



16. Adding Maximum Steps

Never allow an uncontrolled loop.

```
MAX_STEPS = 10
```

Conceptual implementation:

```
for step in range(MAX_STEPS):

    response = call_llm(
        messages,
        tools
    )

    if not response.has_tool_calls():

        return response.text

    execute_tool_calls(
        response.tool_calls
```

```
)  
  
return "Maximum steps reached."
```

This protects against:

- Infinite loops
- Repeated tool calls
- Excessive cost
- Unexpected latency

17. Tool Execution Function

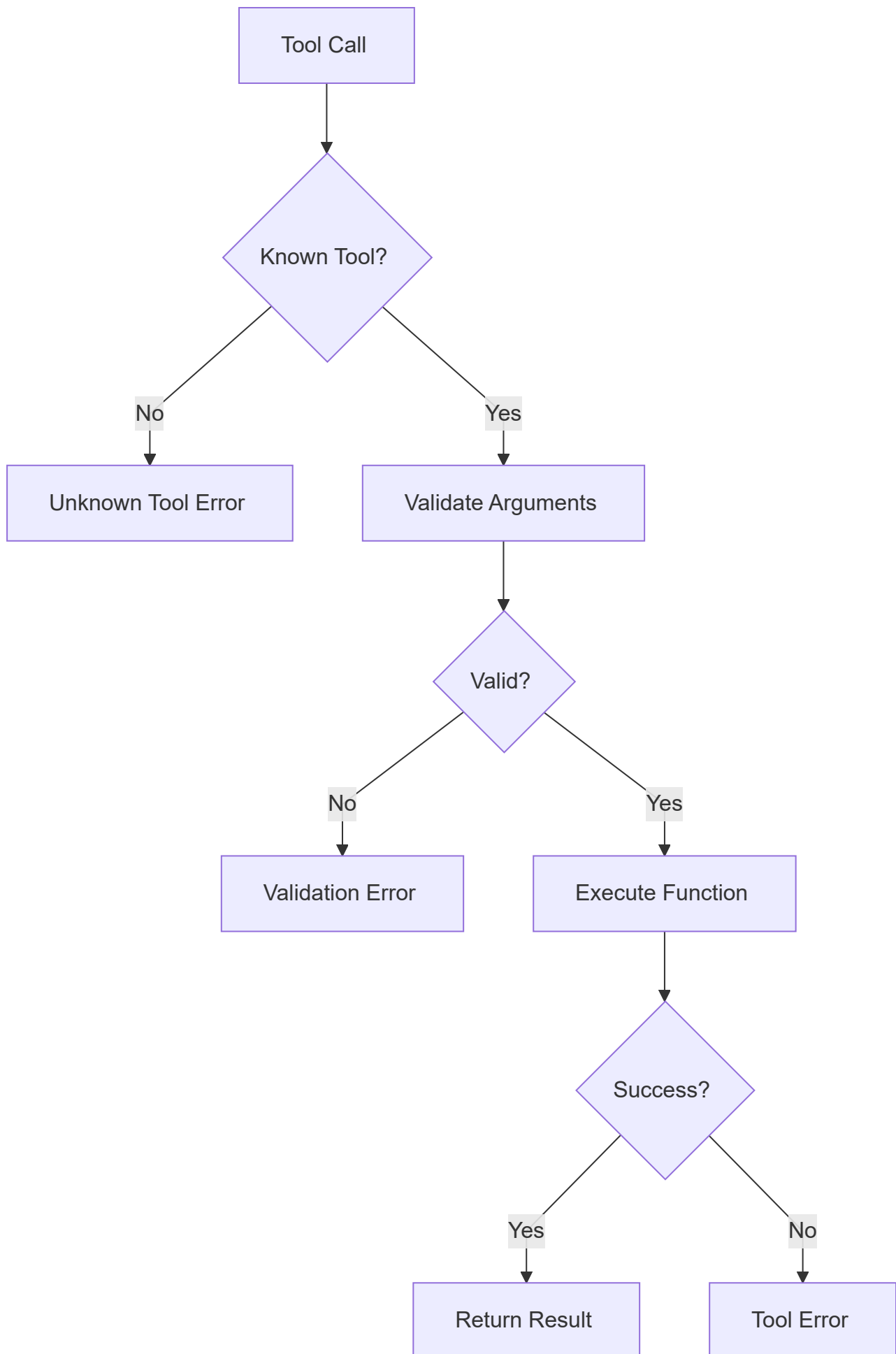
Instead of repeating execution logic, create one function.

```
def execute_tool(  
    tool_name,  
    arguments  
):  
  
    if tool_name not in available_tools:  
  
        return {  
            "error":  
                "Unknown tool"  
        }  
  
    tool_function = available_tools[  
        tool_name  
    ]  
  
    try:
```



```
    return tool_function(  
        **arguments  
    )  
  
except Exception as error:  
  
    return {  
        "error": str(error)  
    }
```

18. Tool Execution Pipeline



19. Structured Tool Results



Prefer structured outputs.

Bad:

```
"Delhi is 32 degrees and rainy."
```

Better:

```
{  
  "city": "Delhi",  
  "temperature": 32,  
  "unit": "celsius",  
  "condition": "rainy"  
}
```

Why?

Because structured results are easier to:

validate

process

store

pass to another tool

use programmatically

20. Tool Errors Should Also Be Structured

Instead of:

```
Something went wrong!!!
```

return:

```
{  
  "success": false,  
  "error_code": "CITY_NOT_FOUND",  
  "message": "City was not found."  
}
```

The agent can reason about this more reliably.

21. Retry Strategy

Not every error should be retried.

Example:

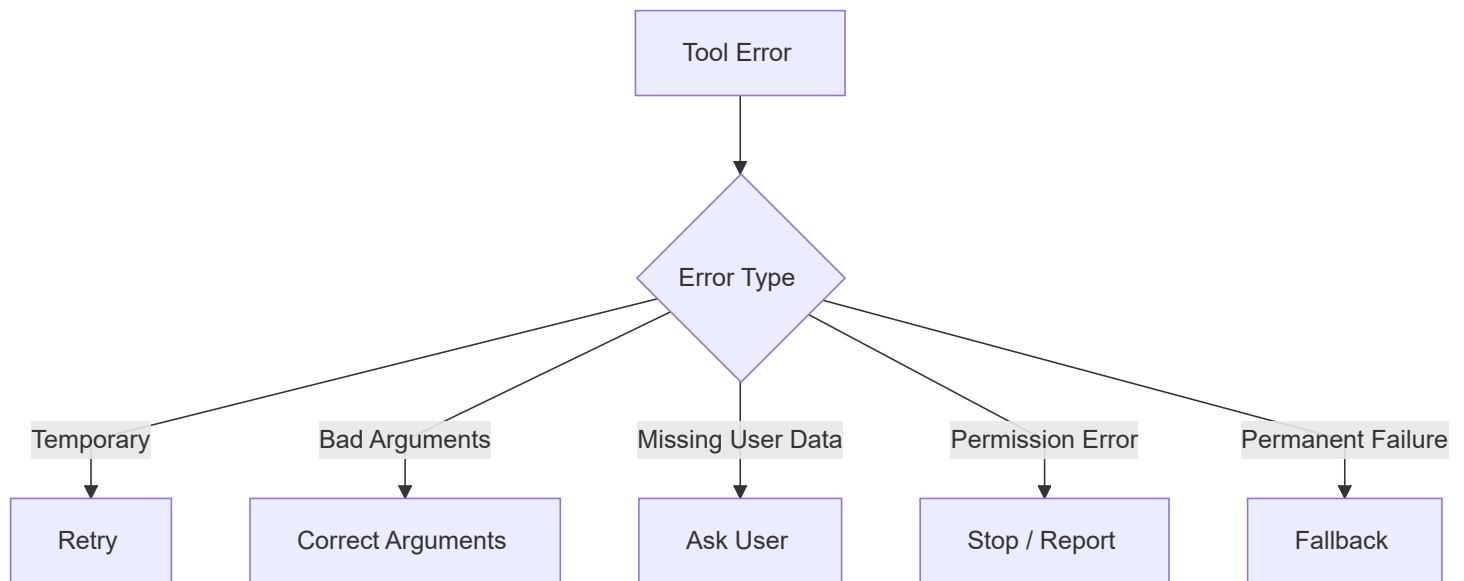
```
API Timeout  
→ Retry may help
```

But:

Invalid User ID

→ Repeating same request won't help

Therefore:



22. Retry Limits

Use:

```
MAX_RETRIES = 2
```

Conceptually:

```
for attempt in range(MAX_RETRIES):  
  
    result = call_tool()  
  
    if result["success"]:  
        break
```

Never create:

Retry forever

23. Tool Timeouts



External tools may hang.

Example:

```
API request
↓
No response
↓
Agent waits forever
```

Use timeouts:

```
Tool Call
↓
Timeout Limit
↓
Success?
✓      ✗
Yes    No
↓      ↓
Use    Handle Error
```

Timeouts improve reliability.

24. Fallback Tools

Sometimes an agent can use an alternative.

Example:

```
Primary Weather API
↓
Fails
↓
Backup Weather API
```

Or:

```
Internal Search
↓
No Result
↓
Web Search
```

This is called:

Fallback Strategy

But only add fallbacks when genuinely useful.

25. Tool Confirmation

For read-only tools:


```
get_weather()
```

```
search_products()
```

```
read_document()
```

we often do not need human confirmation.

But for action tools:

```
send_email()
```

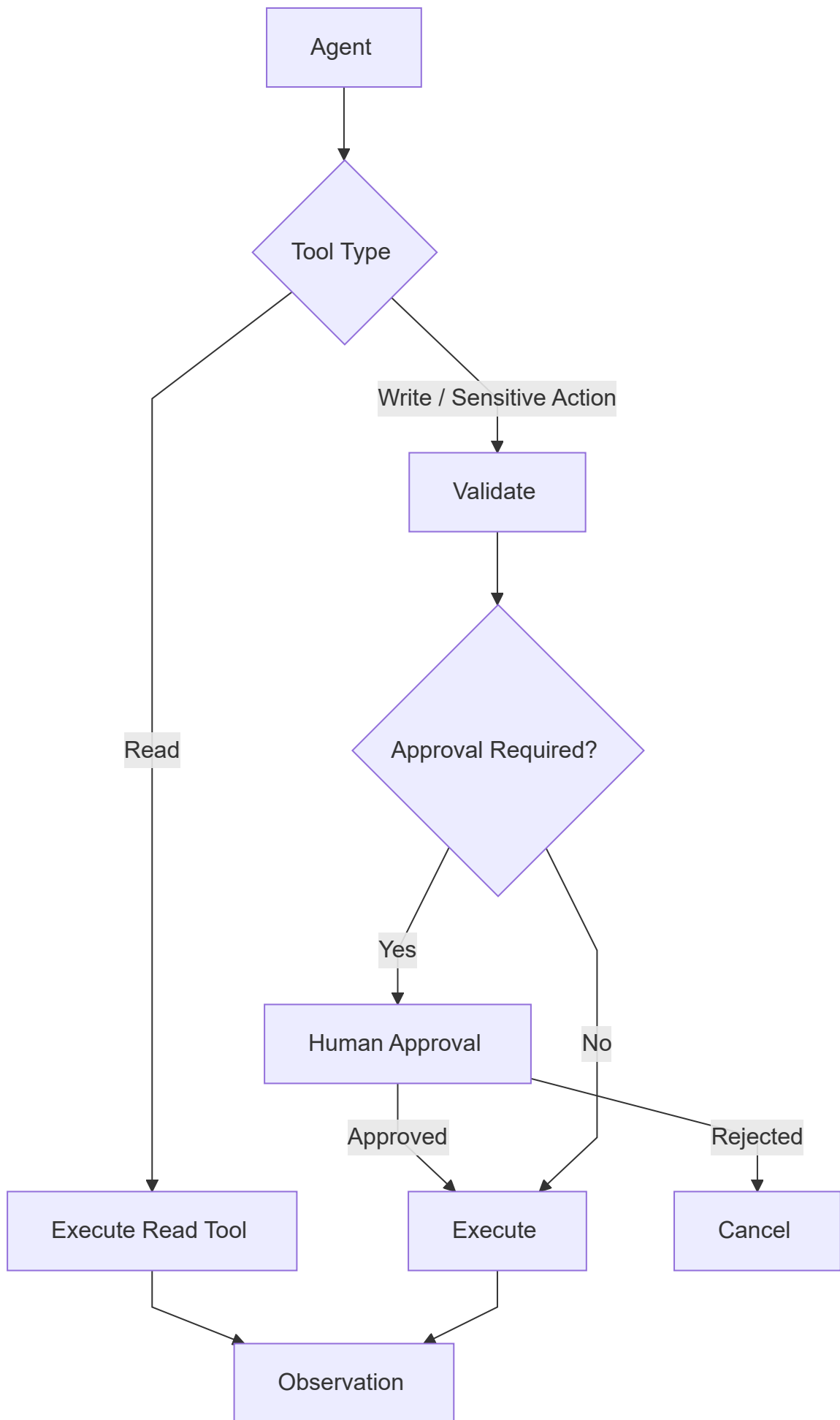
```
make_payment()
```

```
delete_record()
```

```
book_ticket()
```

confirmation may be appropriate.

26. Read vs Write Tool Flow



27. Tool Idempotency



A useful production concept:

Idempotency

An operation is idempotent when repeating the same request does not create unintended duplicate effects.

For example:

Read weather

can safely be repeated.

But:

Send payment

repeating it may cause:

Duplicate Payment

Therefore action tools may require:

Idempotency keys

Duplicate checks

Transaction IDs

You only need the basic idea for now.

28. Tool Permissions

Different agents may receive different tools.

Example:

Support Agent

- ✓ search_order
- ✓ get_delivery_status
- ✓ create_support_ticket

- X refund_payment
- X delete_user

Another authorized agent may have:

Refund Agent

- ✓ get_order
- ✓ verify_refund
- ✓ issue_refund

Do not give every agent every capability.

29. Tool Context

Sometimes a tool needs application context that the LLM should not provide.

Example:

```
get_my_orders()
```

The tool may internally receive:

```
Authenticated User ID
```

from the application session.

Do NOT rely on the LLM to say:

```
user_id = 123
```

for security-sensitive identity.

Better:

```
Authenticated Session
```

↓

```
Backend
```

↓

```
Trusted user_id
```

↓

```
Tool
```

30. Security Boundary



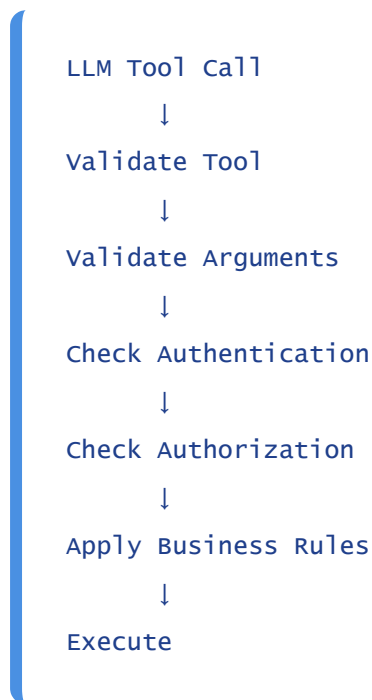
Treat:

LLM Output

as:

Untrusted Input

Therefore:



The LLM should never bypass your backend security.

31. Tool Calling With Real LLM Libraries

You will commonly encounter two implementation styles:

Provider SDK Tool Calling

and:

Framework Tool Calling

For example:

Provider SDK

→ Direct model API

LangChain

→ Tool abstraction

LangGraph

→ Tool + stateful graph workflow

We should understand both conceptually.

32. LangChain Tool Concept

A simple tool can be created using a decorator.

Conceptually:

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get current weather for a city."""

    return f"{city}: 32°C, rainy"
```

The function's:

```
Name

Type hints

Docstring
```

can help define the tool interface.

33. Why Type Hints Matter

Example:

```
def get_weather(
    city: str
) -> str:
```

The framework can understand:

```
city  
→ string
```

For another tool:

```
def calculator(  
    a: float,  
    b: float  
) -> float:
```

it knows:

```
a  
→ number  
  
b  
→ number
```

Clear typing improves schema generation and validation.

34. Creating Multiple Tools

```
@tool  
def get_weather(city: str) -> str:  
    """Get current weather for a city."""  
  
    return f"{city}: 32°C, rainy"  
  
@tool  
def multiply(  
    a: float,  
    b: float  
) -> float:
```

```
"""Multiply two numbers."""
```

```
return a * b
```

Then:

```
tools = [  
    get_weather,  
    multiply  
]
```

35. Binding Tools to an LLM



In frameworks such as LangChain, you may see:

```
llm_with_tools = llm.bind_tools(  
    tools  
)
```

Conceptually:

```
LLM  
+  
Tool Schemas  
↓  
Tool-Aware LLM
```

Important:

```
bind_tools()
```

does **not** mean:

```
Execute all tools.
```

It means the model is informed about the available tool definitions so it can request them.

36. Invoking the Tool-Aware LLM

Conceptually:

```
response = llm_with_tools.invoke(  
    "What is the weather in Delhi?"  
)
```

The response may contain:

```
content:  
""  
  
tool_calls:  
[  
    {  
        "name": "get_weather",  
        "args": {  
            "city": "Delhi"  
        }  
    }  
]
```

The empty text is normal because the model is requesting an action instead of giving the final answer yet.

37. Inspecting Tool Calls

Conceptually:

```
print(  
    response.tool_calls  
)
```

Possible output:

```
[  
  {  
    "name": "get_weather",  
    "args": {  
      "city": "Delhi"  
    },  
    "id": "call_123"  
  }  
)
```

Now the application knows which function to execute.

38. Executing a LangChain Tool

Conceptually:

```
tool_call = response.tool_calls[0]

result = get_weather.invoke(
    tool_call["args"]
)
```

Result:

```
Delhi: 32°C, rainy
```

But we still need to return the observation to the model.

39. ToolMessage

In LangChain-style message workflows, tool results can be represented using:

```
ToolMessage
```

Conceptually:

```
from langchain_core.messages import ToolMessage
```

Then:

```
tool_message = ToolMessage(
    content=str(result),
    tool_call_id=tool_call["id"]
)
```

This associates:

Tool Result

with the original:

Tool Call

40. Message History

The conversation may now contain:

HumanMessage

↓

AIMessage containing tool call

↓

ToolMessage containing result

Then send all of this back to the model.

Human:

Weather in Delhi?

AI:

Call get_weather("Delhi")

Tool:

32°C, rainy

AI:

The current weather in Delhi is...

41. Why Preserve the AI Tool-Call Message?



A common mistake is:

```
User Message
↓
Tool Result
```

while dropping the model's tool-call request.

The correct conversation history generally needs to preserve the relationship:

```
User Message
↓
AI Tool Request
↓
Tool Result
↓
AI Final Response
```

This lets the model understand why the tool result exists.

42. Simplified LangChain-Style Implementation

```
from langchain_core.tools import tool
from langchain_core.messages import (
    HumanMessage,
    ToolMessage
)

@tool
def get_weather(city: str) -> str:
    """Get current weather for a city."""

    return f"{city}: 32°C, rainy"

tools = [
    get_weather
]

llm_with_tools = llm.bind_tools(
    tools
)

messages = [
    HumanMessage(
        content=
            "What is the weather in Delhi?"
    )
]

# First model call
```

```

response = llm_with_tools.invoke(
    messages
)

messages.append(
    response
)

# Execute requested tools
for tool_call in response.tool_calls:

    if tool_call["name"] == "get_weather":

        result = get_weather.invoke(
            tool_call["args"]
        )

        messages.append(
            ToolMessage(
                content=str(result),
                tool_call_id=
                    tool_call["id"]
            )
        )

# Second model call
final_response = (
    llm_with_tools.invoke(
        messages
    )
)

print(
    final_response.content
)

```

Exact library APIs can evolve.

Understand the architecture rather than memorizing syntax.

43. Problem With Manual Tool Execution

Our code currently manually does:

```
if tool_call["name"] == "get_weather":
```

With many tools:

```
if tool == "weather":  
    ...  
  
elif tool == "calculator":  
    ...  
  
elif tool == "search":  
    ...  
  
elif tool == "database":  
    ...
```

This becomes repetitive.

A tool registry solves this.

44. Tool Registry With Framework Tools

```
tools = [  
    get_weather,  
    multiply  
]  
  
tools_by_name = {  
    tool.name: tool  
    for tool in tools  
}
```

Then:

```
selected_tool = tools_by_name[  
    tool_call["name"]  
]  
  
result = selected_tool.invoke(  
    tool_call["args"]  
)
```

This works regardless of which registered tool was selected.

45. Generic Multiple Tool Execution

```
for tool_call in response.tool_calls:
```

```

tool_name = tool_call["name"]

selected_tool = tools_by_name.get(
    tool_name
)

if selected_tool is None:

    result = {
        "error":
            "Unknown tool"
    }

else:

    try:

        result = selected_tool.invoke(
            tool_call["args"]
        )

    except Exception as error:

        result = {
            "error":
                str(error)
        }

messages.append(
    ToolMessage(
        content=str(result),
        tool_call_id=
            tool_call["id"]
    )
)

```

Now the execution layer is generic.

46. Complete Tool-Calling Agent Loop



Instead of only allowing one round of tool calls:

```
MAX_STEPS = 10

for step in range(MAX_STEPS):

    response = llm_with_tools.invoke(
        messages
    )

    messages.append(
        response
    )

    # No tool requested
    if not response.tool_calls:

        print(
            response.content
        )

        break

    # Execute requested tools
    for tool_call in response.tool_calls:

        selected_tool = tools_by_name.get(
            tool_call["name"]
        )
```

```

if selected_tool is None:

    result = {
        "error":
            "Unknown tool"
    }

else:

    try:

        result = selected_tool.invoke(
            tool_call["args"]
        )

    except Exception as error:

        result = {
            "error":
                str(error)
        }

messages.append(
    ToolMessage(
        content=str(result),

        tool_call_id=
            tool_call["id"]
    )
)

```

Now:

```
LLM
↓
Tool
↓
LLM
↓
Tool
↓
LLM
↓
...
↓
Final Answer
```

can happen automatically until the model stops requesting tools.

47. This Is Already a Basic Agent

Notice what we now have:

```
User Goal
      ↓
    LLM
      ↓
Tool Selection
      ↓
Tool Execution
      ↓
Observation
      ↓
Message / State Update
      ↓
    LLM Again
      ↓
Repeat Until Finished
```


That is already the foundation of a:

Tool-Calling AI Agent

48. Why Do We Need LangGraph Then?



You might ask:

If we can create an agent using a Python loop, why do we need LangGraph?

Because real workflows may need:

Multiple branches

Complex state

Persistence

Checkpointing

Human approval

Retries

Different nodes

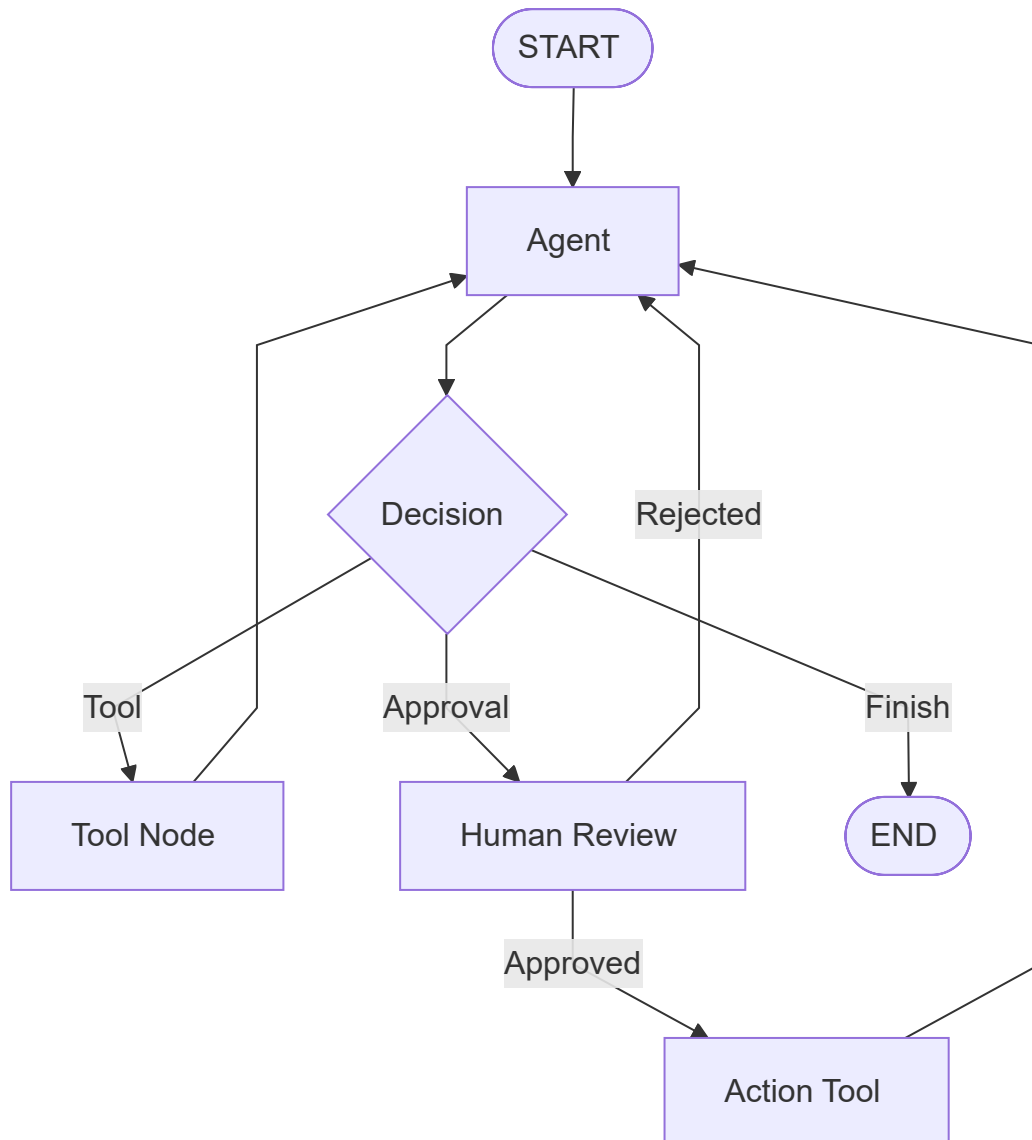
Conditional routing

Pause / resume

Long-running workflows

A manual loop becomes harder to manage.

Example:



LangGraph gives us an explicit structure for such workflows.

We will study that in Chapter 4.

49. Tool Calling vs LangGraph

Do not confuse them.

Tool Calling

→ Model chooses external functions.

LangGraph

→ Orchestrates stateful workflows containing models, tools, branches and loops.

LangGraph can use tool calling internally.

50. Tool Calling vs MCP

We will study MCP later, but remember this basic distinction:

Tool Calling

→ How the model requests a tool.

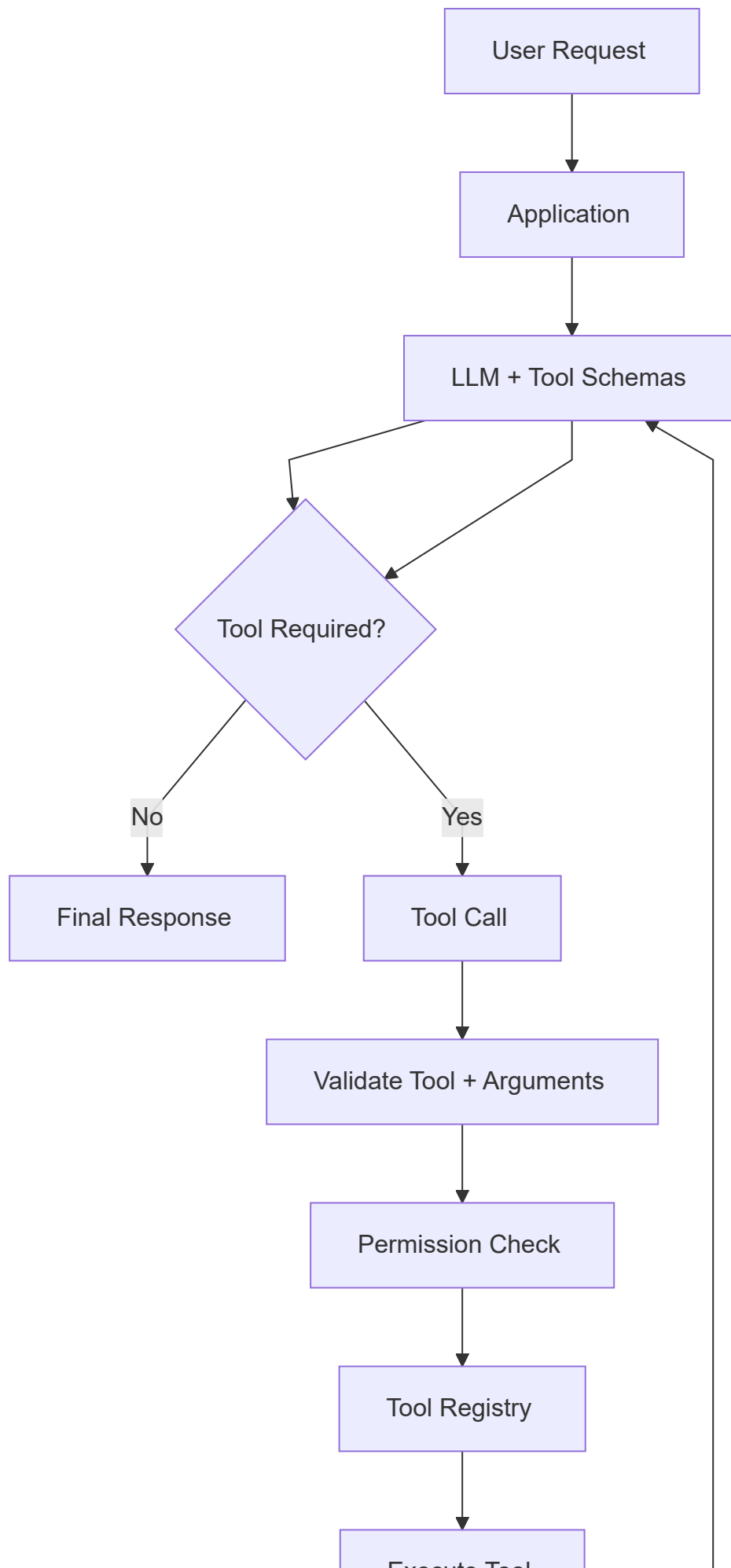
MCP

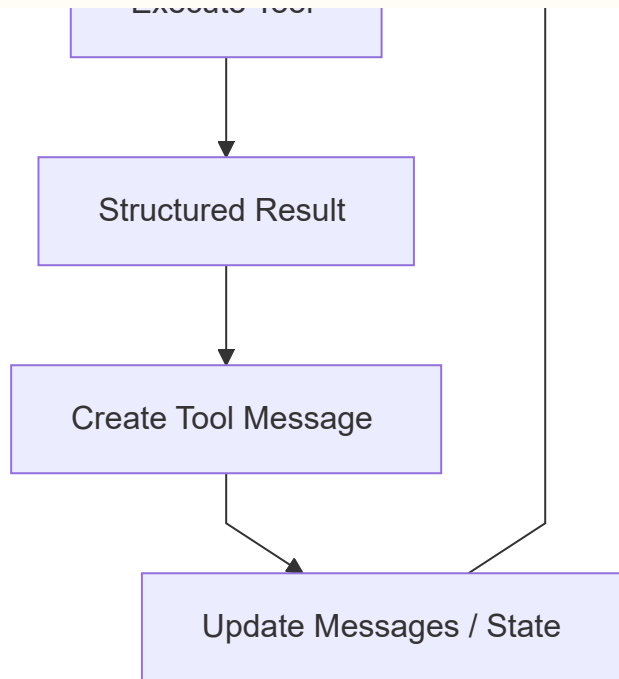
→ A standardized protocol/interface for exposing tools and context to AI applications.

They are related but solve different problems.

51. Complete Tool-Calling Architecture







This architecture captures most of what you need to understand about tool calling.

52. Practical Tool Design Checklist

Before exposing a function as a tool, check:

- ☐ Does the agent actually need this tool?
- ☐ Is the name clear?
- ☐ Is the description clear?
- ☐ Are parameters strongly defined?
- ☐ Are inputs validated?
- ☐ Is output structured?
- ☐ Are errors structured?
- ☐ Is authentication handled?

- ❑ Is authorization handled?
- ❑ Does it modify real data?
- ❑ Does it require human approval?
- ❑ Does it have a timeout?
- ❑ Are retries limited?
- ❑ Could duplicate execution cause damage?

53. Common Interview Questions

Q1. How do you implement multiple tools for an LLM? ★ ★ ★ ★ ★

Answer:

I define each tool with a clear schema, provide the available tool definitions to the LLM, maintain a mapping between tool names and actual functions, inspect the model's tool calls, validate and execute the selected functions, and return their results to the model.

Q2. What is a tool registry? ★ ★ ★ ★ ★

Answer:

A tool registry is a mapping between tool names and their actual executable functions, allowing the application to dynamically locate and execute the tool selected by the LLM.

Q3. Sequential vs parallel tool calls?



Answer:

Sequential calls are required when one tool depends on the result of another. Independent tool calls can potentially execute in parallel to reduce latency.

Q4. What happens if a tool fails?

Answer:

The application should return a structured error and determine whether the failure is recoverable. Depending on the error, the system may retry with limits, correct arguments, use a fallback, ask the user or terminate gracefully.

Q5. Why should retries be limited?

Answer:

Unlimited retries can create infinite loops, excessive latency, unnecessary API usage and unexpected cost.

Q6. What is a tool timeout?

Answer:

A tool timeout limits how long the application waits for a tool to complete before treating the operation as failed.

Q7. What is a fallback tool?

Answer:

A fallback tool is an alternative capability that can be used when the primary tool fails or cannot provide the required information.

Q8. What is the difference between read and write tools? ★ ★ ★ ★

Answer:

Read tools retrieve information without modifying external state, while write or action tools change data or trigger real-world actions and therefore generally require stricter validation and permissions.

Q9. What is least privilege in tool-based agents? ★ ★ ★ ★

Answer:

Least privilege means providing an agent only the tools and permissions necessary for its intended task rather than giving it unrestricted access to external systems.

Q10. Why shouldn't the LLM provide authenticated user IDs?

Answer:

Identity and authorization information should come from trusted application context such as an authenticated session or token. Model-generated identity values should not be trusted for security-sensitive operations.

Q11. What does `bind_tools()` conceptually do? ★ ★ ★ ★ ★

Answer:

It provides the model with the definitions or schemas of available tools so the model can decide when to request them and generate appropriate arguments. It does not itself mean that every tool is executed.

Q12. What is a ToolMessage?

Answer:

A ToolMessage represents the result of a tool execution and associates that result with the corresponding tool-call request so the model can use the observation in its next decision.

Q13. Why preserve tool-call messages in conversation history?

Answer:

The tool-call request and its result form a connected interaction. Preserving both gives the model the context required to understand why a tool was executed and what the returned result represents.

Q14. Why use structured tool outputs?

Answer:

Structured outputs are easier to validate, process, store, combine with other tools and use programmatically than unstructured text.

Q15. Why use LangGraph if a Python agent loop works? ★ ★ ★ ★ ★

Answer:

A simple Python loop works well for basic agents, but LangGraph becomes useful when workflows require explicit state management, branching, cycles, persistence, checkpointing, human-in-the-loop interactions and more complex orchestration.

54. Final Mental Model

Remember these layers:

Layer 1

FUNCTION

`get_weather(city)`



Layer 2

TOOL

Name

+

Description

+

Schema

+

Function



Layer 3

TOOL CALLING

LLM



Select Tool



Generate Arguments



Application Executes



Observation



Layer 4

TOOL-CALLING AGENT

Goal



LLM



Tool



Observation



State Update



LLM



Another Tool?

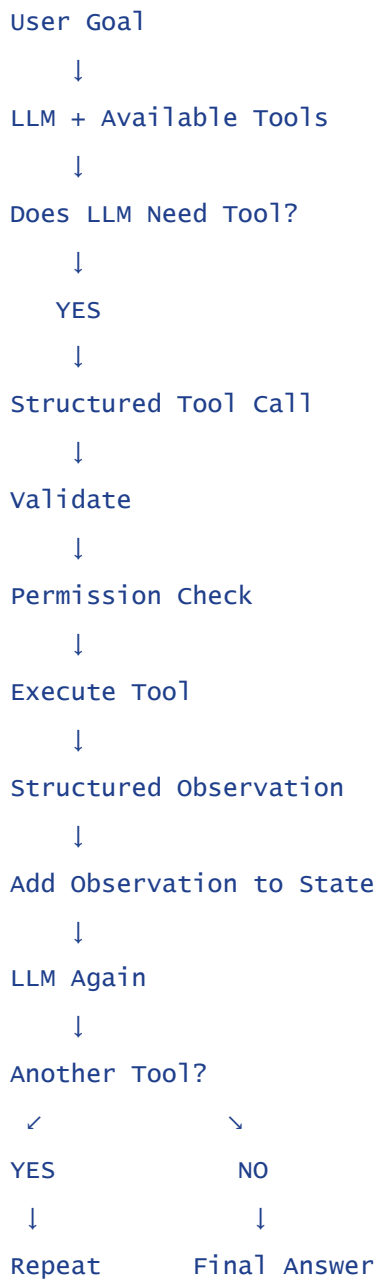


...



Final Answer

The complete loop:



The most important practical principle:

LLM
→ Makes flexible decisions

Tool
→ Performs capability

Application
→ Controls execution

Backend

→ Enforces security

State

→ Tracks results

Loop

→ Enables multi-step behavior

A robust tool-calling agent does not simply allow an LLM to execute arbitrary functions. The model selects from carefully designed tools, while the application validates, authorizes, executes and records every action in a controlled loop.

End of Chapter 2 – Tool Calling
